



湖南大学
HUNAN UNIVERSITY

第四章 排序

湖南大学计算机学院

目录

第一节

比较排序

第二节

线性时间排序

排序问题



输入：数组 $A[1...n]$

输出：把数组A按升序排列并用 $B[1...n]$ 表示

示例： $A=[7,2,5,5,9.6]$ $B=[2, 5, 5, 7, 9.6]$

怎样能更有效率呢？

为什么要排序



- 明显的应用

- 组织MP3库维护
- 电话目录

- 一旦项目被排序，问题就变得简单

- 找到中位数，或找到最接近的对
- 二分搜索，识别统计异常值

- 不明显的应用

- 数据压缩：排序发现重复项
- 计算机图形学：从前到后渲染场景

插入排序

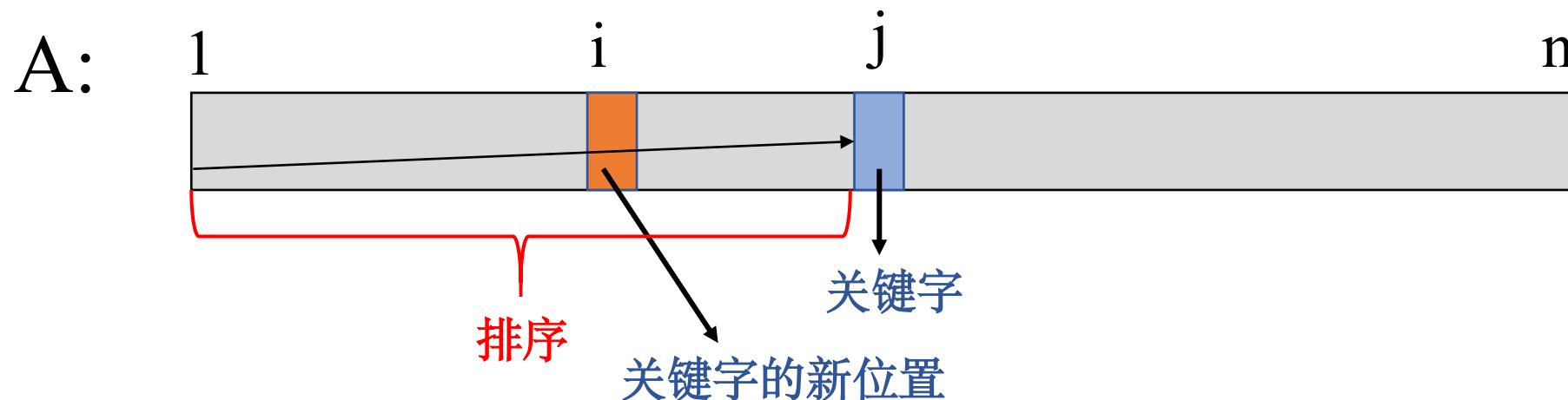


INSERTION-SORT(A, n) 对数组 $A[1..n]$ 进行操作

for $j \leftarrow 2$ to n

将关键字 $A[j]$ 插入到已排序的子数组 $A[1..j-1]$ 中，通过成对的键交换，使其下降到正确的位置

迭代 j 的说明



插入排序伪代码



插入排序

```
1  输入：一个包含  $n$  个元素的数组  $A[1...n]$ 
2  输出：按非降序排列的数组  $A$ 
```

```
3  //依次插入元素
4  for  $j = 2$  to  $n$  do
5       $key \leftarrow A[j]$ 
6       $i \leftarrow j - 1$ 
7      //找到相应位置插入
8      while  $i > 0$  and  $A[i] > key$  do
9           $A[i + 1] \leftarrow A[i]$ 
10          $i \leftarrow i - 1$ 
11      $A[i + 1] \leftarrow key$ 
```

插入排序例子



插入排序

执行次数

7 20 13 2 11 9 12 1

1 输入：一个包含 n 个元素的数组 $A[1...n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 **for** $j = 2$ **to** n **do**

5 $key \leftarrow A[j]$

6 $i \leftarrow j - 1$

7 //找到相应位置插入

8 **while** $i > 0$ and $A[i] > key$ **do**

9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$

插入排序例子



插入排序

执行次数

1 输入：一个包含 n 个元素的数组 $A[1\dots n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 **for** $j = 2$ **to** n **do** 1

5 $key \leftarrow A[j]$ 1

6 $i \leftarrow j - 1$ 1

7 //找到相应位置插入

8 **while** $i > 0$ **and** $A[i] > key$ **do** 1

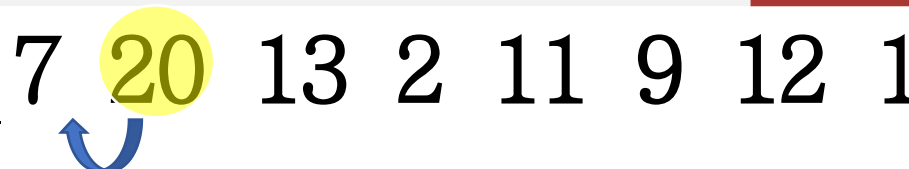
9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$ 1

合计

5



插入排序例子



插入排序

1 输入：一个包含 n 个元素的数组 $A[1...n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 **for** $j = 2$ **to** n **do**

5 $key \leftarrow A[j]$

6 $i \leftarrow j - 1$

7 //找到相应位置插入

8 **while** $i > 0$ **and** $A[i] > key$ **do**

9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$

合计

执行次数

1

1

1

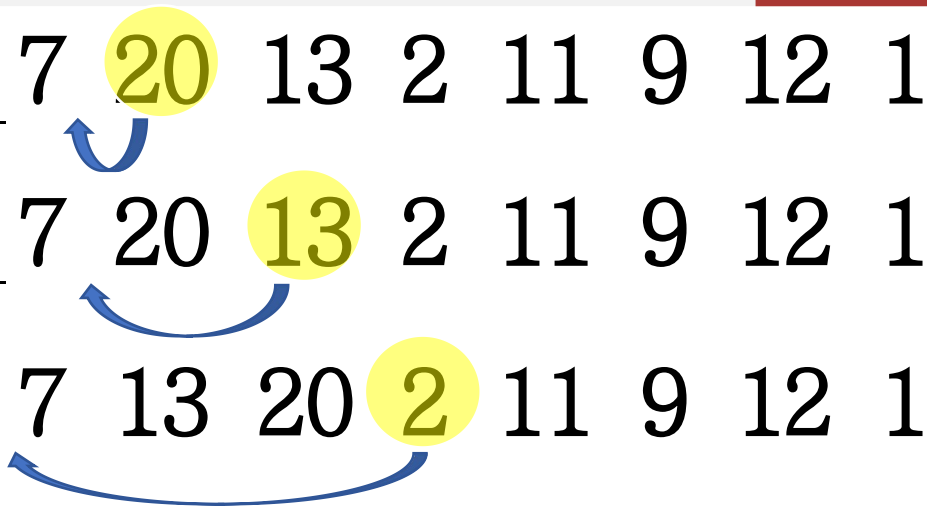
4

3

3

1

14



插入排序例子



插入排序

1 输入：一个包含 n 个元素的数组 $A[1...n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 for $j = 2$ to n do

5 $key \leftarrow A[j]$

6 $i \leftarrow j - 1$

7 //找到相应位置插入

8 while $i > 0$ and $A[i] > key$ do

9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$

执行次数

1

1

1

3

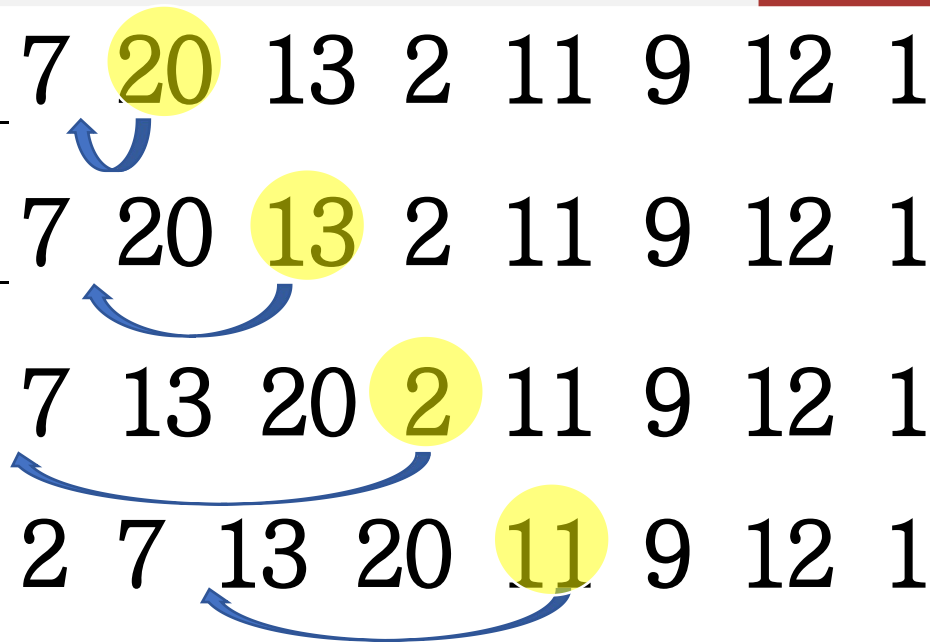
2

2

1

合计

11



插入排序例子



插入排序	执行次数	
1 输入：一个包含 n 个元素的数组 $A[1...n]$		7 20 13 2 11 9 12 1
2 输出：按非降序排列的数组 A		7 20 13 2 11 9 12 1
3 //依次插入元素		
4 for $j = 2$ to n do	1	7 13 20 2 11 9 12 1
5 $key \leftarrow A[j]$	1	
6 $i \leftarrow j - 1$	1	2 7 13 20 11 9 12 1
7 //找到相应位置插入		
8 while $i > 0$ and $A[i] > key$ do	4	2 7 11 13 20 9 12 1
9 $A[i + 1] \leftarrow A[i]$	3	
10 $i \leftarrow i - 1$	3	
11 $A[i + 1] \leftarrow key$	1	
合计	14	

插入排序例子



插入排序

1 输入：一个包含 n 个元素的数组 $A[1...n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 for $j = 2$ to n do

5 $key \leftarrow A[j]$

6 $i \leftarrow j - 1$

7 //找到相应位置插入

8 while $i > 0$ and $A[i] > key$ do

9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$

执行次数

1

1

1

3

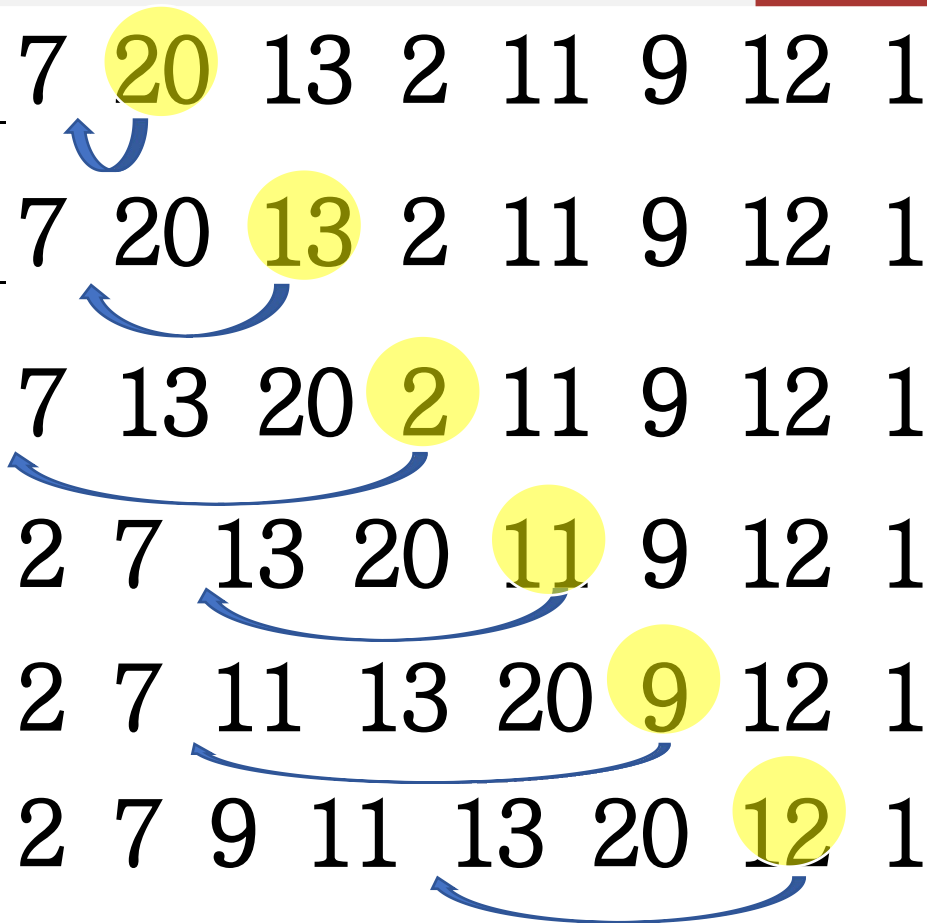
2

2

1

合计

11



插入排序例子



插入排序

1 输入：一个包含 n 个元素的数组 $A[1...n]$

2 输出：按非降序排列的数组 A

3 //依次插入元素

4 for $j = 2$ to n do

5 $key \leftarrow A[j]$

6 $i \leftarrow j - 1$

7 //找到相应位置插入

8 while $i > 0$ and $A[i] > key$ do

9 $A[i + 1] \leftarrow A[i]$

10 $i \leftarrow i - 1$

11 $A[i + 1] \leftarrow key$

合计

执行次数

1

1

1

8

7

7

1

26

7 20 13 2 11 9 12 1

7 20 13 2 11 9 12 1

7 13 20 2 11 9 12 1

2 7 13 20 11 9 12 1

2 7 11 13 20 9 12 1

2 7 9 11 13 20 12 1

2 7 9 11 12 13 12 1

插入排序例子



排序结果 1 2 7 9 11 12 13 12

共计执行了 $5 + 8 + 14 + 11 + 14 + 11 + 26 = 91$ 条语句

插入排序分析



对于插入分析，伪代码各行与其对应常量假设如下：

1	输入：一个包含 n 个元素的数组 $A[1..n]$	
2	输出：按非降序排列的数组 A	代价
3	//依次插入元素	
4	for $j = 2$ to n do	c_4
5	$key \leftarrow A[j]$	c_5
6	$i \leftarrow j - 1$	c_6
7	//找到相应位置插入	
8	while $i > 0$ and $A[i] > key$ do	c_8
9	$A[i + 1] \leftarrow A[i]$	c_9
10	$i \leftarrow i - 1$	c_{10}
11	$A[i + 1] \leftarrow key$	c_{11}

插入排序分析



对于插入分析，该算法总的运行时间是每一条语句执行时间之和，执行一条语句需要 c_i 时间，又共执行了 n_i 次，那么它在总运行时间中占 $c_i \times n_i$ 。

对于插入排序每次循环，假设 t_j 为第 8 条语句中 while 循环所做的判断次数。注意第 8 条语句会比第 9 和第 10 条语句多执行一次，因为循环条件最终总会判断失败才跳出。

对每一对“语句时间”与“执行次数”之积求和，得：

$$\begin{aligned} T(n) = & c_4(n-1) + c_5(n-1) + c_6(n-1) + c_8 \sum_{j=2}^n t_j \\ & + c_9 \sum_{j=2}^n (t_j - 1) + c_{10} \sum_{j=2}^n (t_j - 1) + c_{11}(n-1) \end{aligned}$$

插入排序最好情况分析



在插入排序中最佳情况 是若输入数组已经是排好序的。此时，对每个 $j = 2, 3, \dots, n$ ，当 i 取其初始值 $j - 1$ 时都有 $A[i] \leq \text{key}$ 。于是 $t_j = 1$ ，因此

$$\begin{aligned} T(n) &= (c_4 + c_5 + c_6 + c_8 + c_{11})(n - 1) \\ &= an + b \end{aligned}$$

即最好运行时间是 n 的线性函数，可表示为 $an + b$ ，也就是 $O(n)$

插入排序最坏情况分析



在插入排序中最坏情况 是若输入数组完全逆序。此时，每个元素 $A[j]$ 都必须与整个已排序子数组 $A[1..j-1]$ 中的每一个元素比较。此时 $t_j = j$ ，得

$$\begin{aligned} T(n) &= (c_4 + c_5 + c_6 + c_{11})(n-1) + c_8 \left(\frac{n(n+1)}{2} - 1 \right) + (c_9 + c_{10}) \frac{n(n-1)}{2} \\ &= \left(\frac{c_8 + c_9 + c_{10}}{2} \right) n^2 + \left(c_4 + c_5 + c_6 + \frac{c_8}{2} - \frac{c_9}{2} - \frac{c_{10}}{2} + c_{11} \right) n - (c_4 + c_5 + c_6 + c_8 + c_{11}) \end{aligned}$$

即最坏运行时间是 n 的二次函数，可表示为 $an^2 + bn + c$ ，也就是 $O(n^2)$

插入排序分析



实际中，我们通常不会深入到每条语句逐一计数，只需关注核心操作的执行次数。对插入排序而言，核心操作是元素的**比较**与**移动**：

- 在最好情况下，每轮仅需 1 次比较且不移动元素，总比较次数为 $n - 1$ ，总移动次数为 0，复杂度为 $O(n)$ 。
- 在最坏情况下，第 j 个元素需要与前面 $j - 1$ 个元素逐一比较并移动，总比较次数与总移动次数均为 $\sum_{j=2}^n j - 1 = \frac{n(n-1)}{2}$ ，复杂度为 $O(n^2)$ 。
- 在平均情况下，每个元素平均需要移动已排序部分一半的元素，总比较与移动次数仍为二次阶，复杂度为 $O(n^2)$ 。

插入排序



BINARY-INSERTION-SORT(A, n)

▷ $A[1..n]$

for $j \leftarrow 2$ to n

将关键字 $A[j]$ 插入到已排序的子数组 $A[1..j-1]$ 中的正确位置, 使用二分搜索找到合适的位置

二分搜索需要的时间是 $O(\log n)$ 。然而, 插入后移动元素仍然需要 $O(n)$ 的时间。

复杂性: $O(n \log n)$ 次比较

$O(n^2)$ 次交换

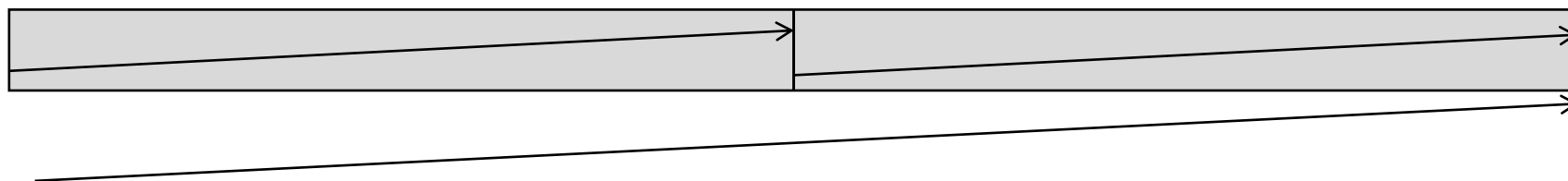
归并排序



分
治
算
法

归并排序 $A[1 \dots n]$

- 1、如果 $n=1$ ，则完成（无需排序）
- 2、否则，递归地对 $A[1 \dots n/2]$ 和 $A[n/2+1 \dots n]$ 进行排序
- 3、“合并”两个已排序的子数组



关键子程序：归并

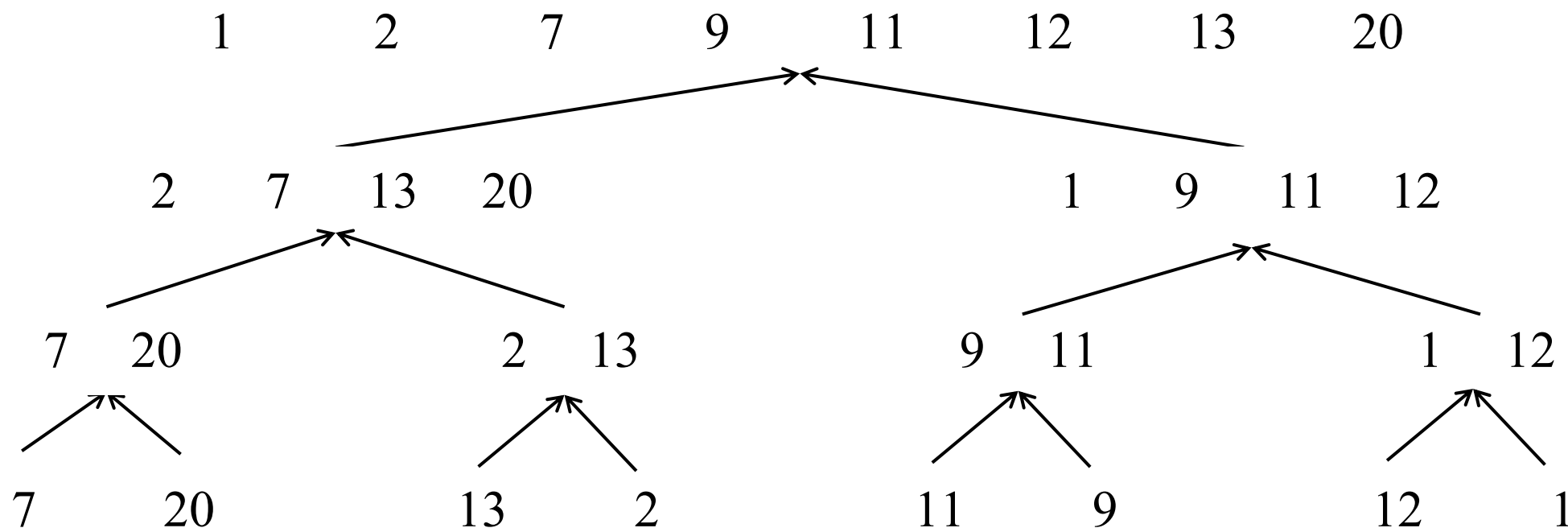
归并排序



第3层合并:

第2层合并:

第1层合并:



合并两个已排序的数组



20 12

13 11

7 9

2 1

合并两个已排序的数组



20 12

13 11

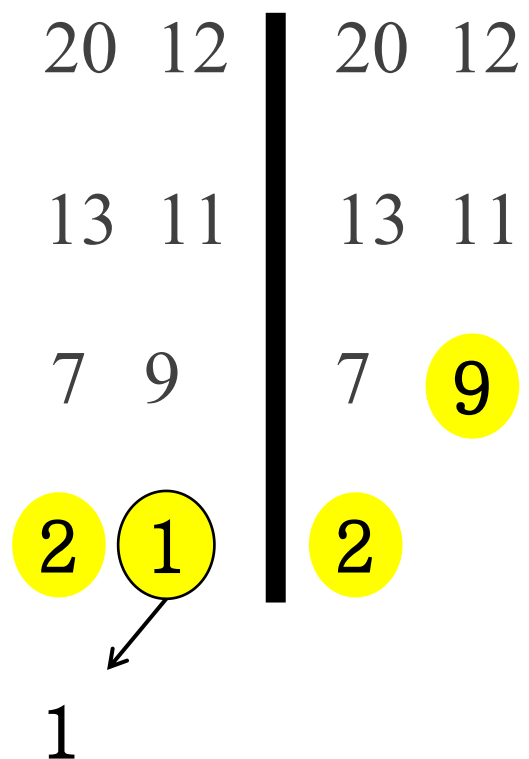
7 9

2 1

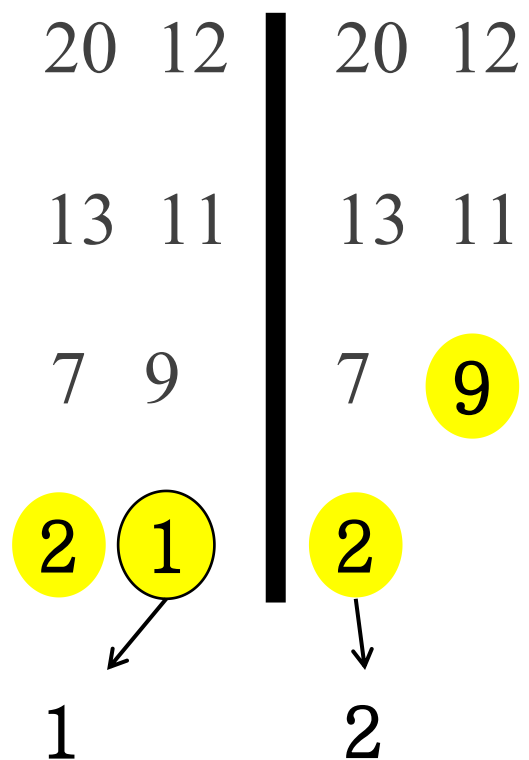


1

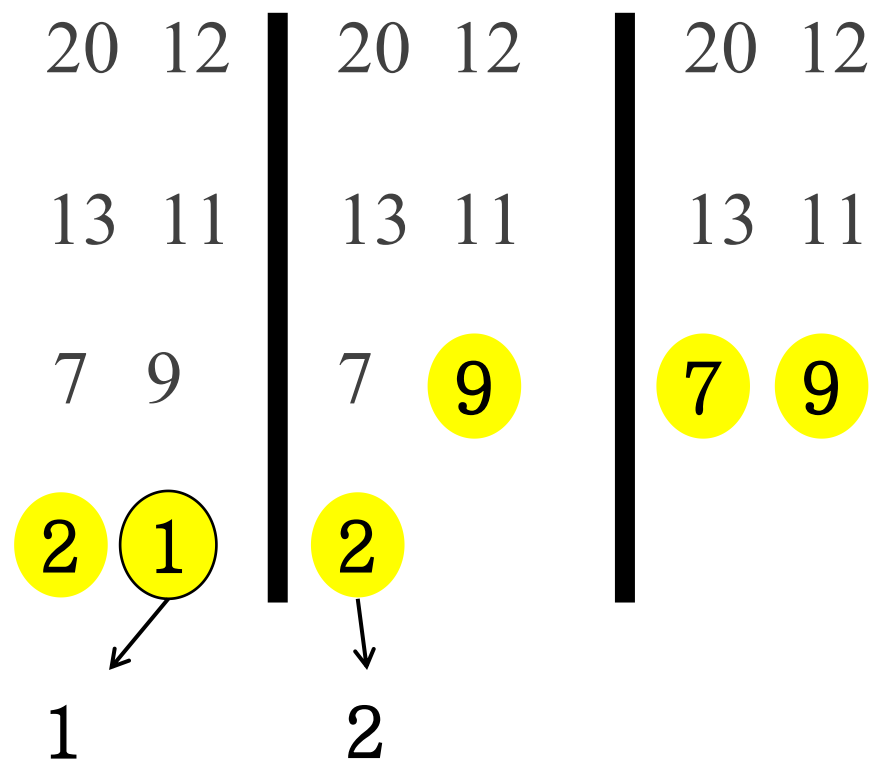
合并两个已排序的数组



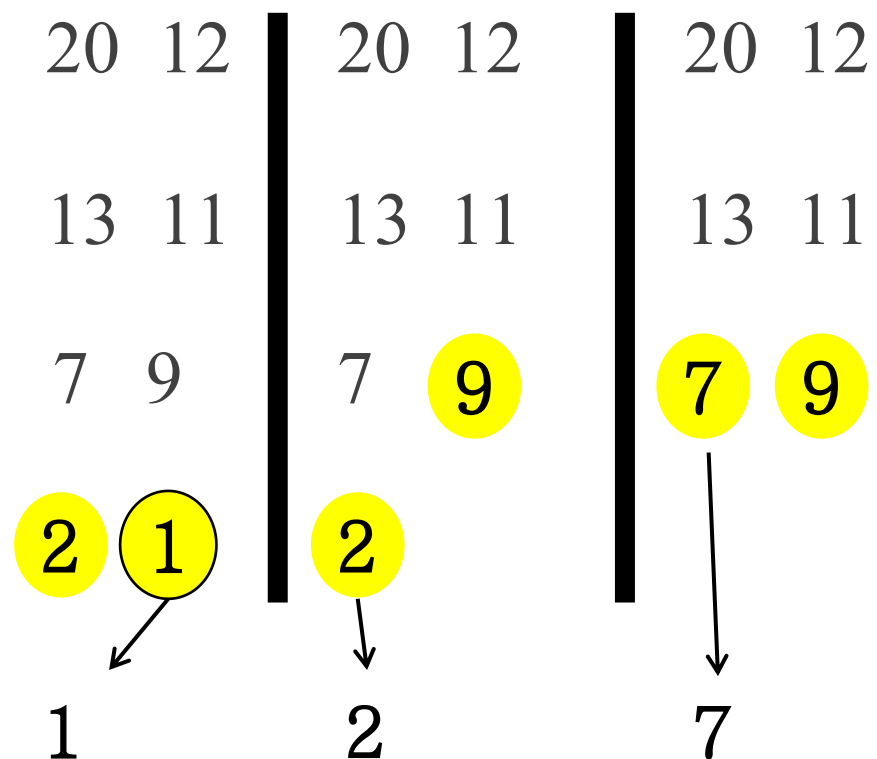
合并两个已排序的数组



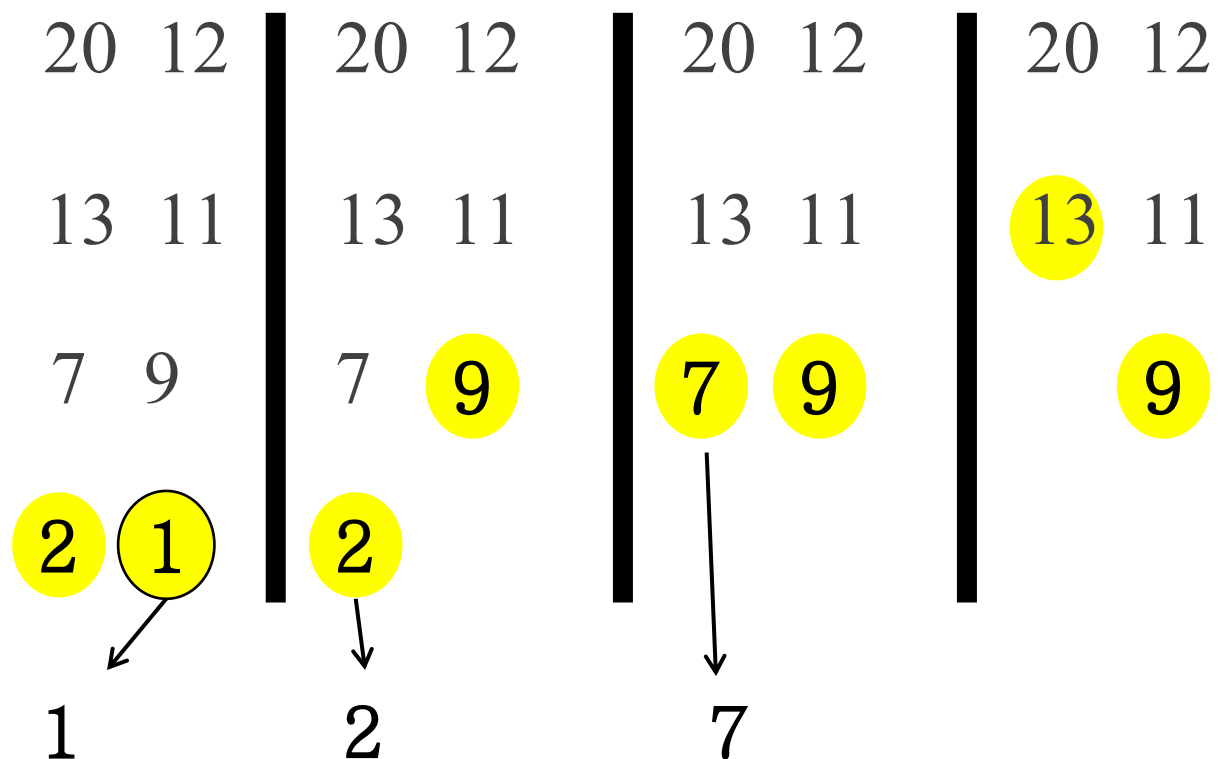
合并两个已排序的数组



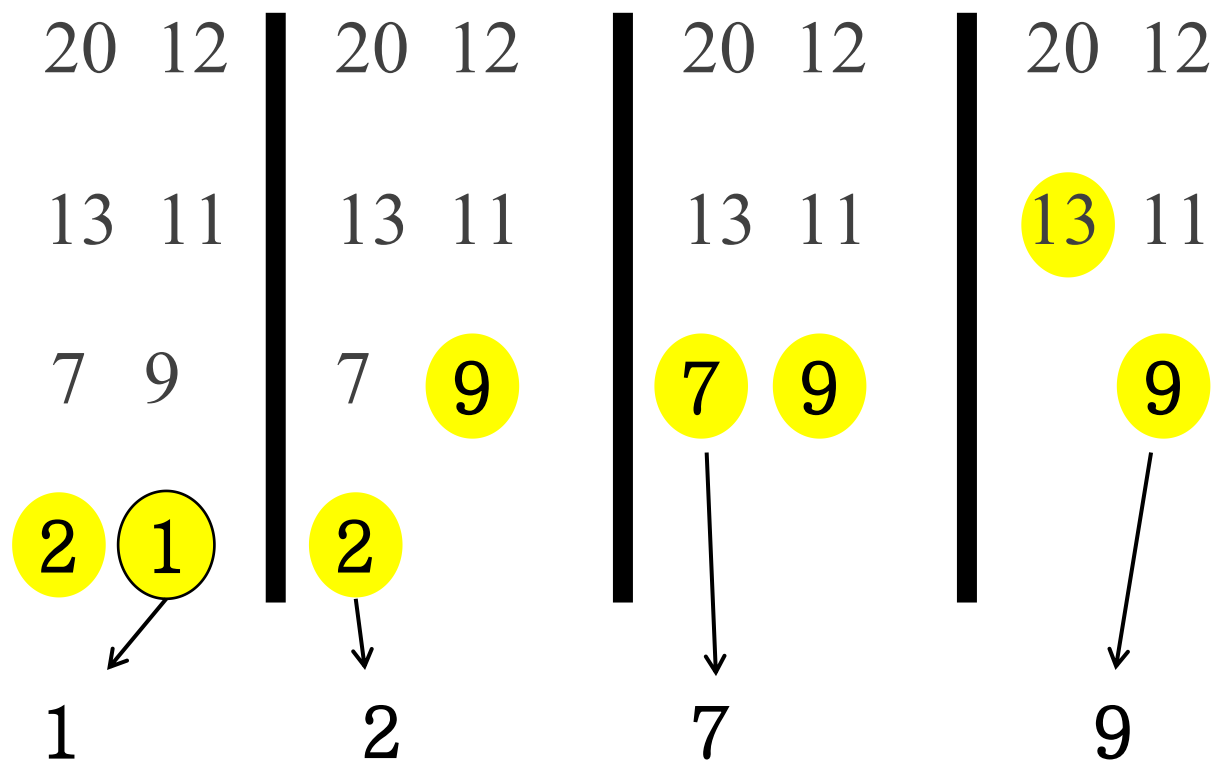
合并两个已排序的数组



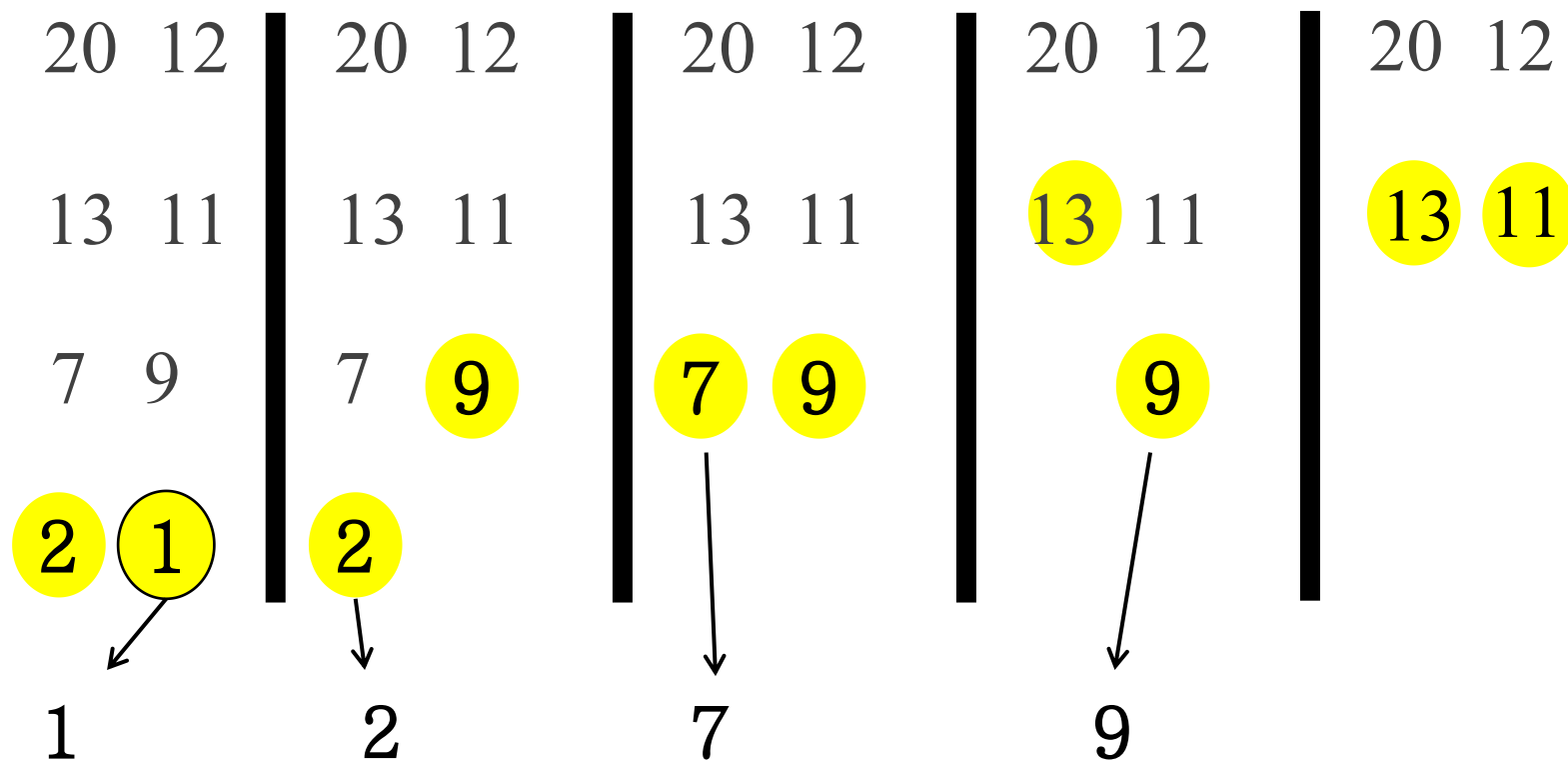
合并两个已排序的数组



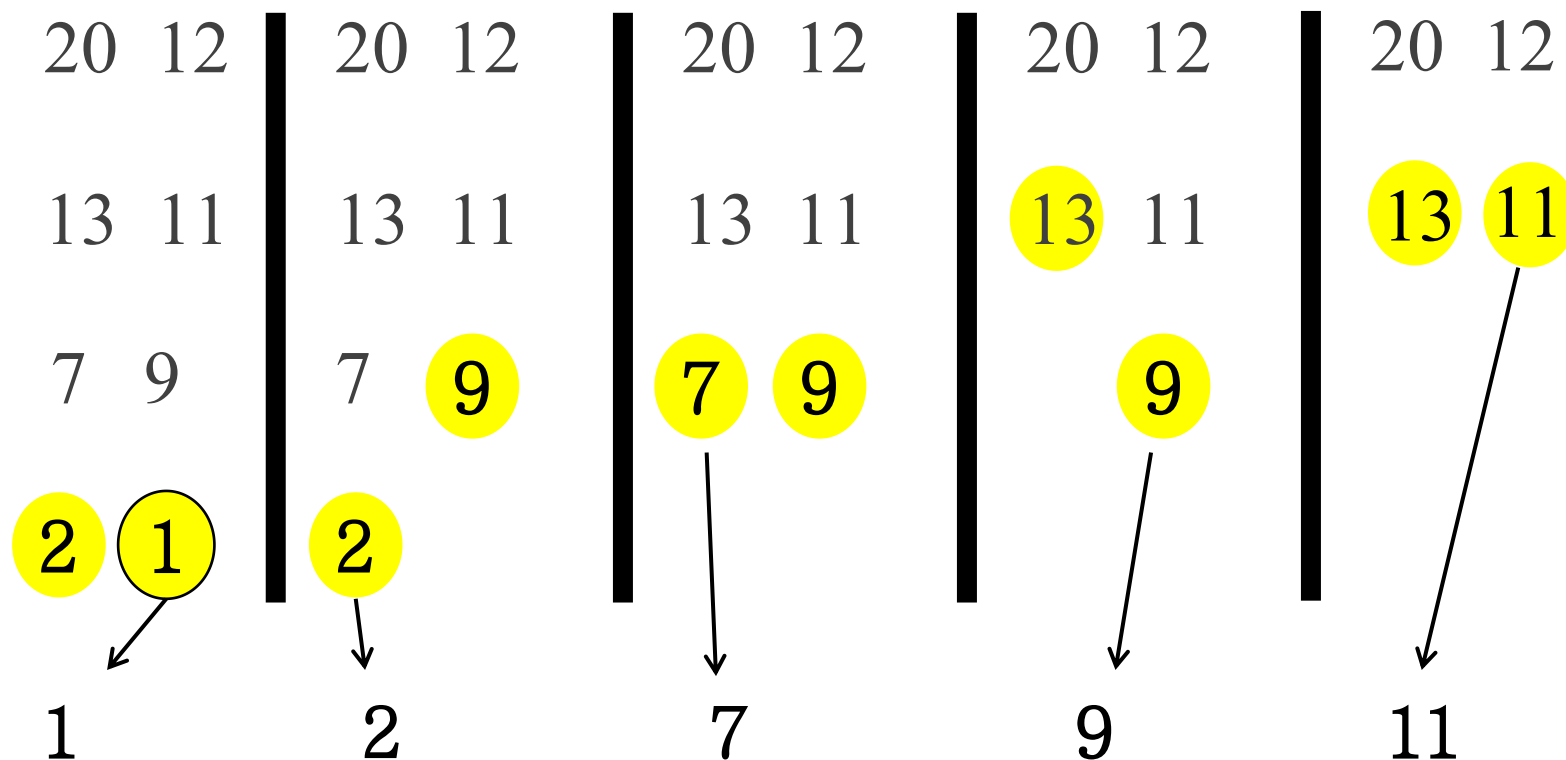
合并两个已排序的数组



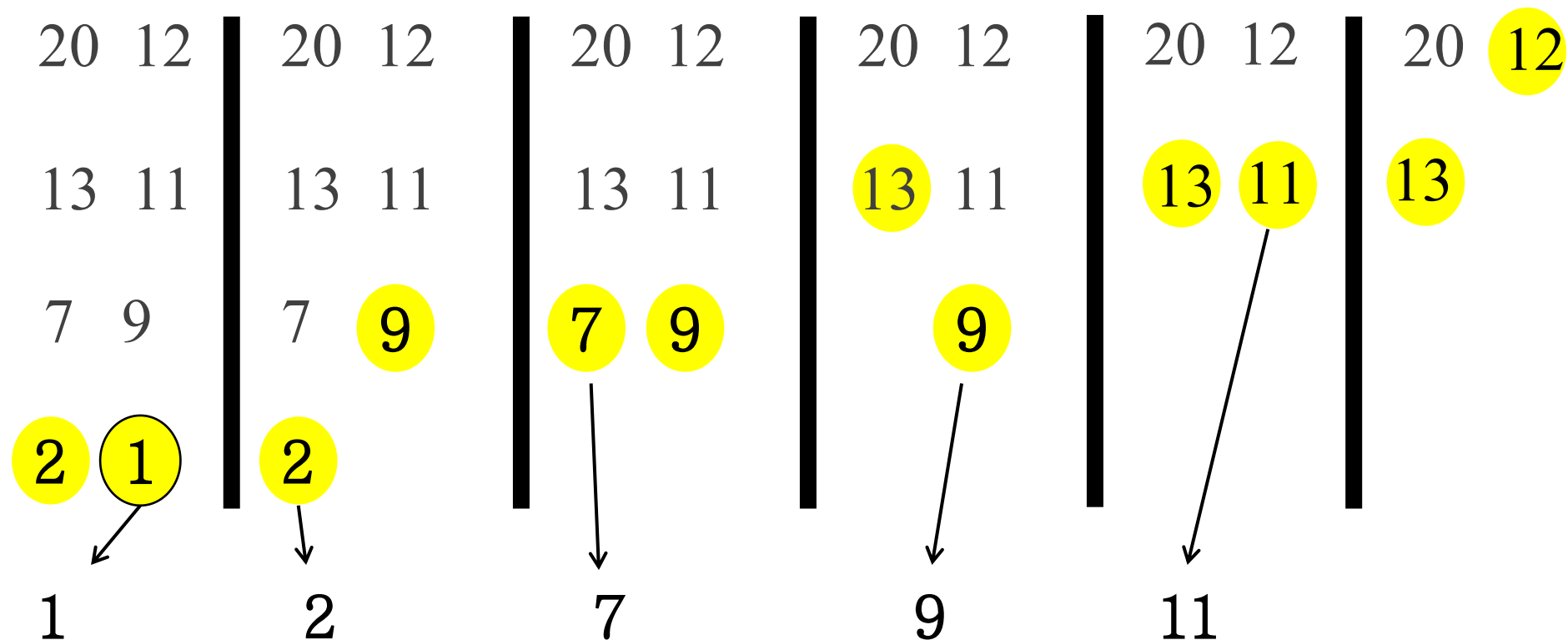
合并两个已排序的数组



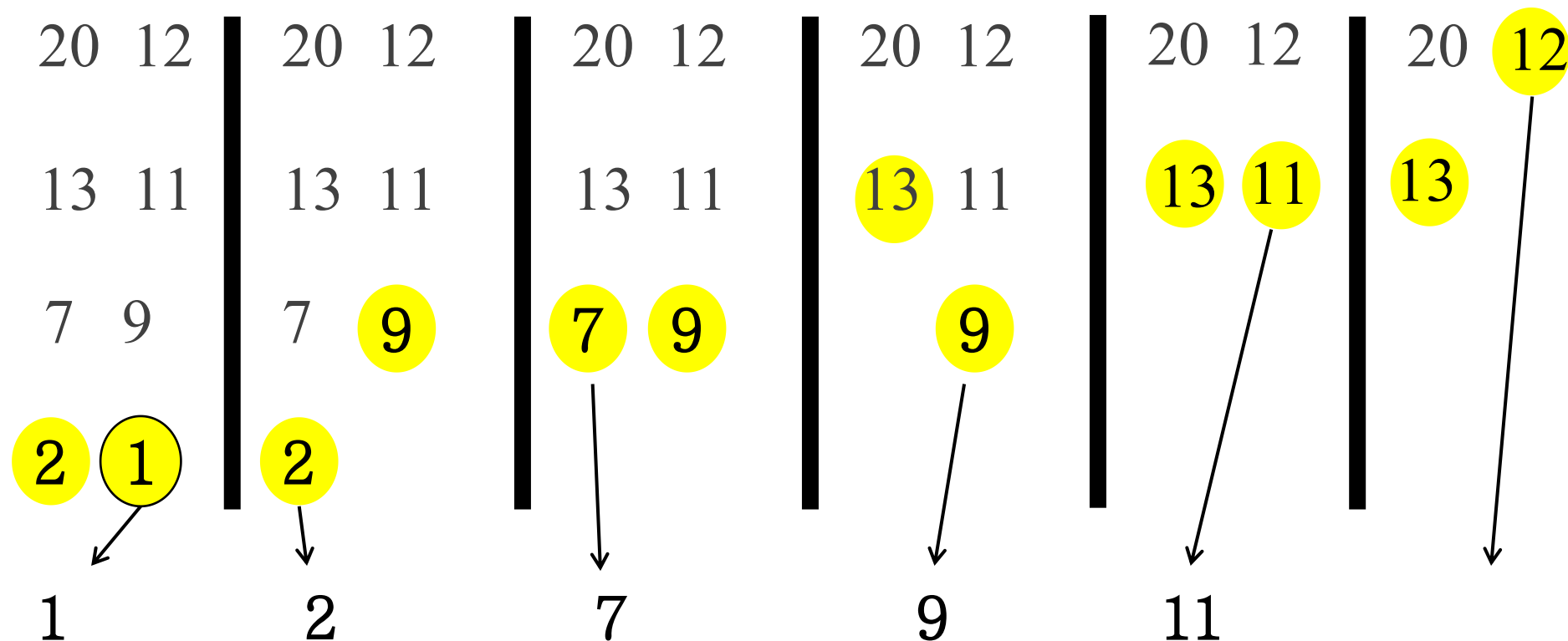
合并两个已排序的数组



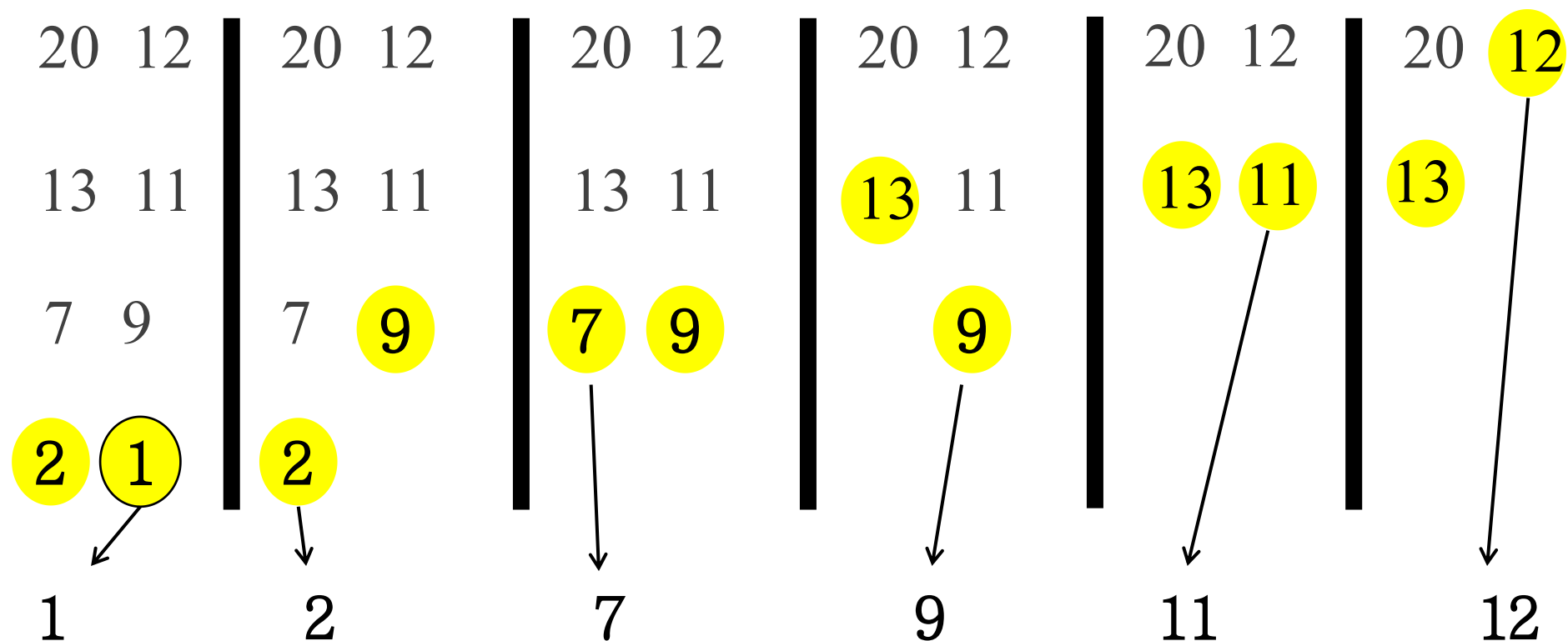
合并两个已排序的数组



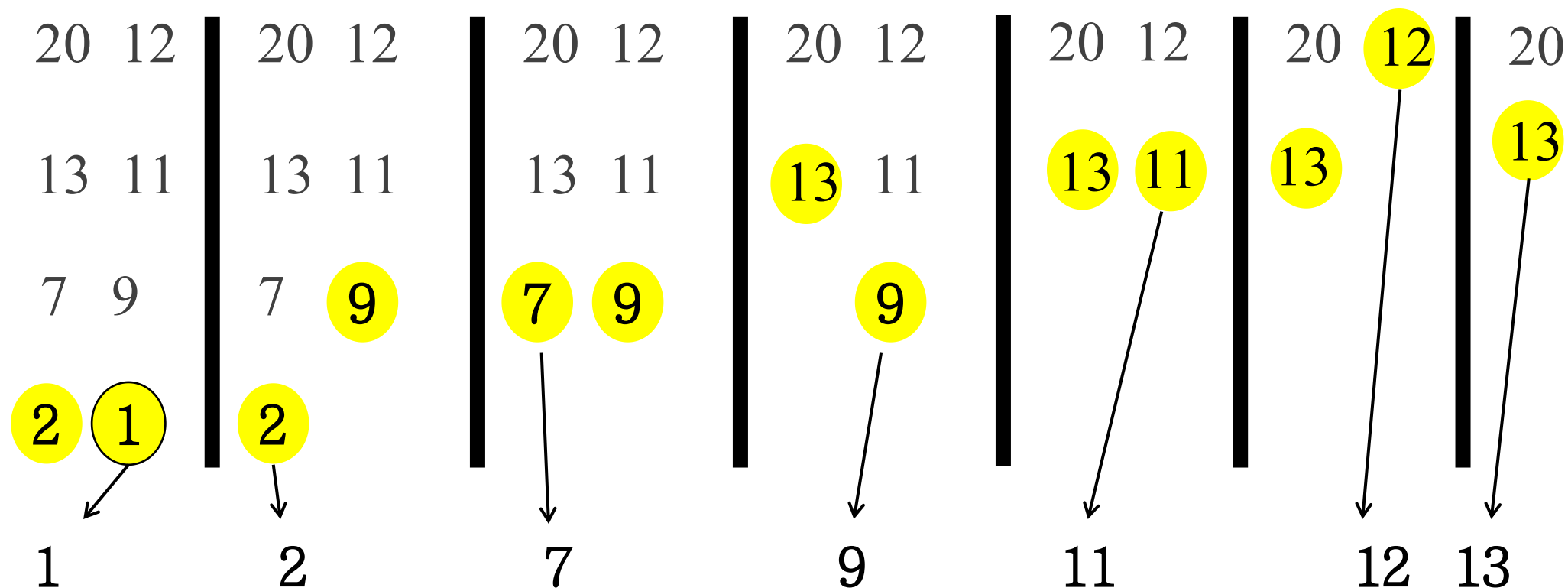
合并两个已排序的数组



合并两个已排序的数组



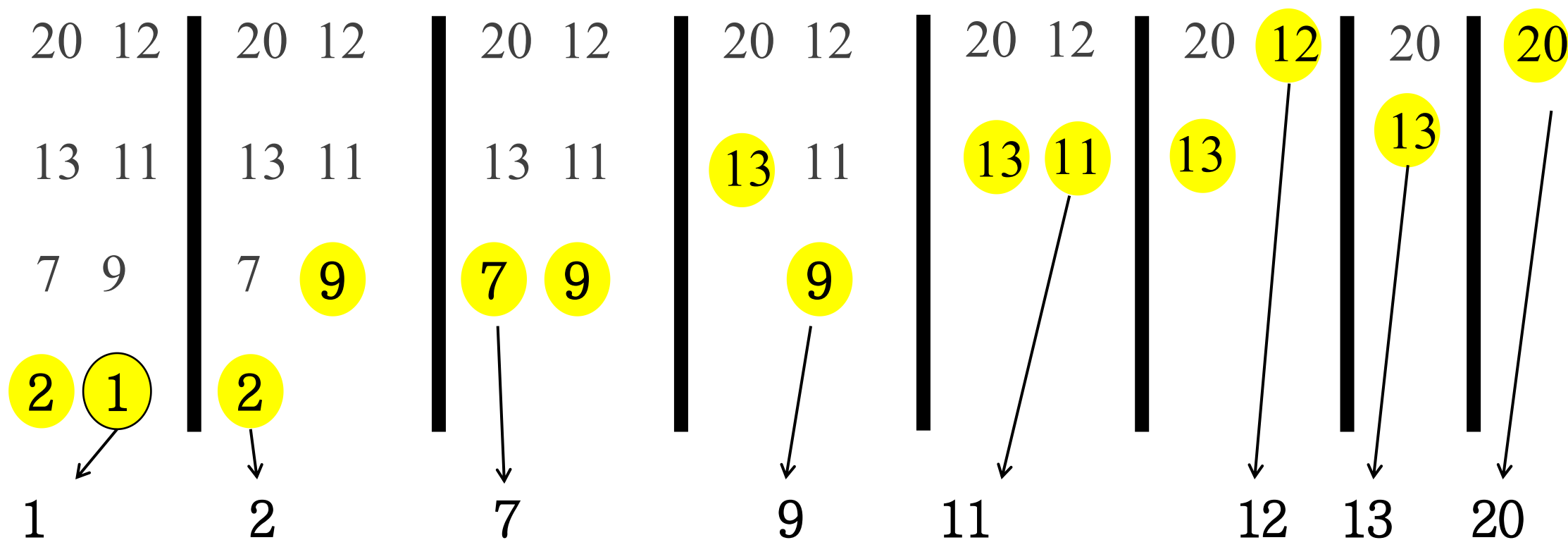
合并两个已排序的数组



合并两个已排序的数组



排序
结果



这个算法需要 $O(n)$ 的时间来合并两个大小为 n 的已排序数组（线性时间）

分析归并排序



归并排序 $A[1..n]$

1. 如果 $n=1$, 则完成

2. 递归地对 $A[1..n/2]$ 和 $A[n/2+1..n]$ 进行排序

3. “合并”两个已排序的列表

$T(n)$

$O(1)$

$2T(n/2)$

$O(n)$

$$T(n) = \begin{cases} O(1) & \text{if } n=1 \\ 2T(n/2) + O(n) & \text{if } n>1 \end{cases}$$

$T(n) = ?$

递归求解



解决 $T(n)=2T(n/2)+cn$, 其中 $c>0$ 是常数。

递归树



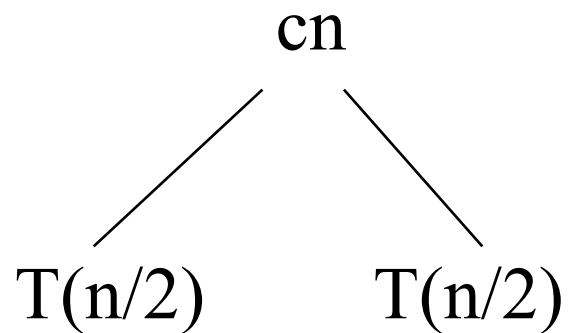
解决 $T(n)=2T(n/2)+cn$, 其中 $c>0$ 是常数。

$T(n)$

递归树



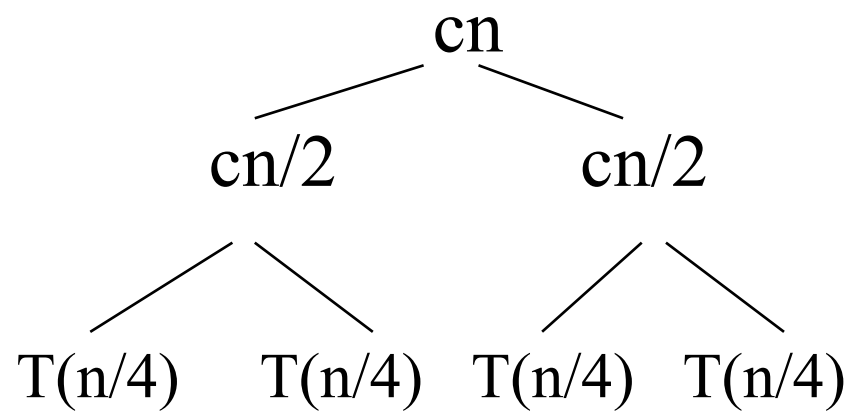
解决 $T(n)=2T(n/2)+cn$ ，其中 $c>0$ 是常数。



递归树



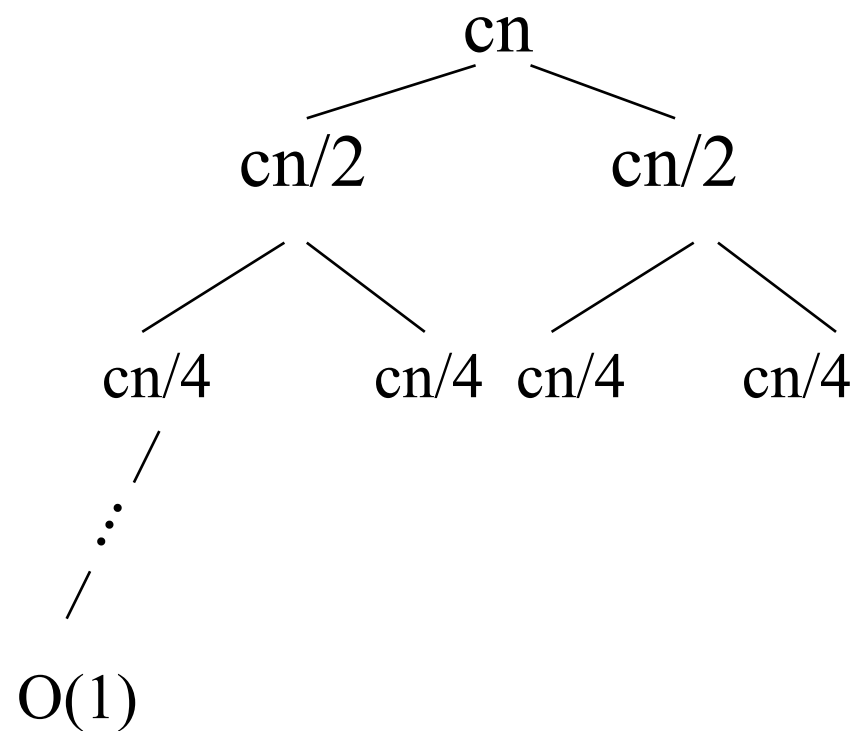
解决 $T(n)=2T(n/2)+cn$, 其中 $c>0$ 是常数。



递归树



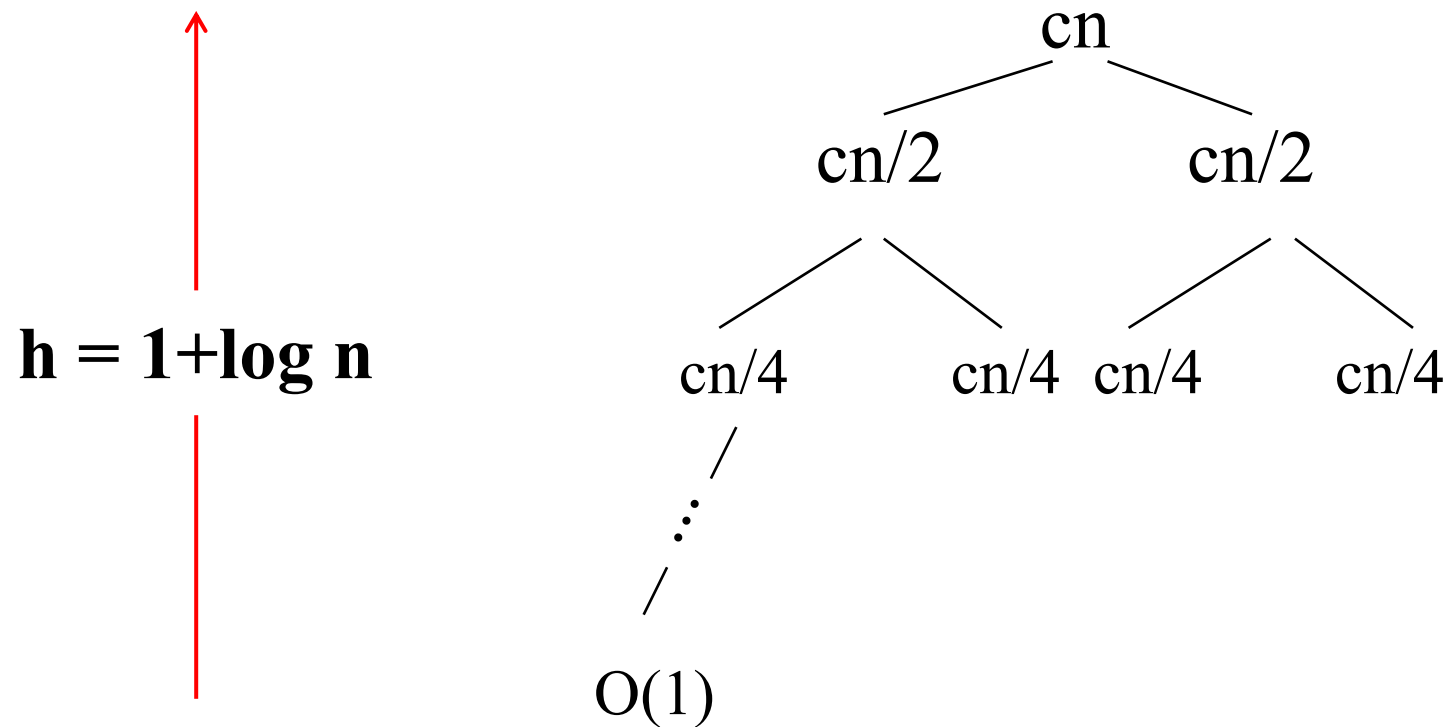
解决 $T(n)=2T(n/2)+cn$ ，其中 $c>0$ 是常数。



递归树



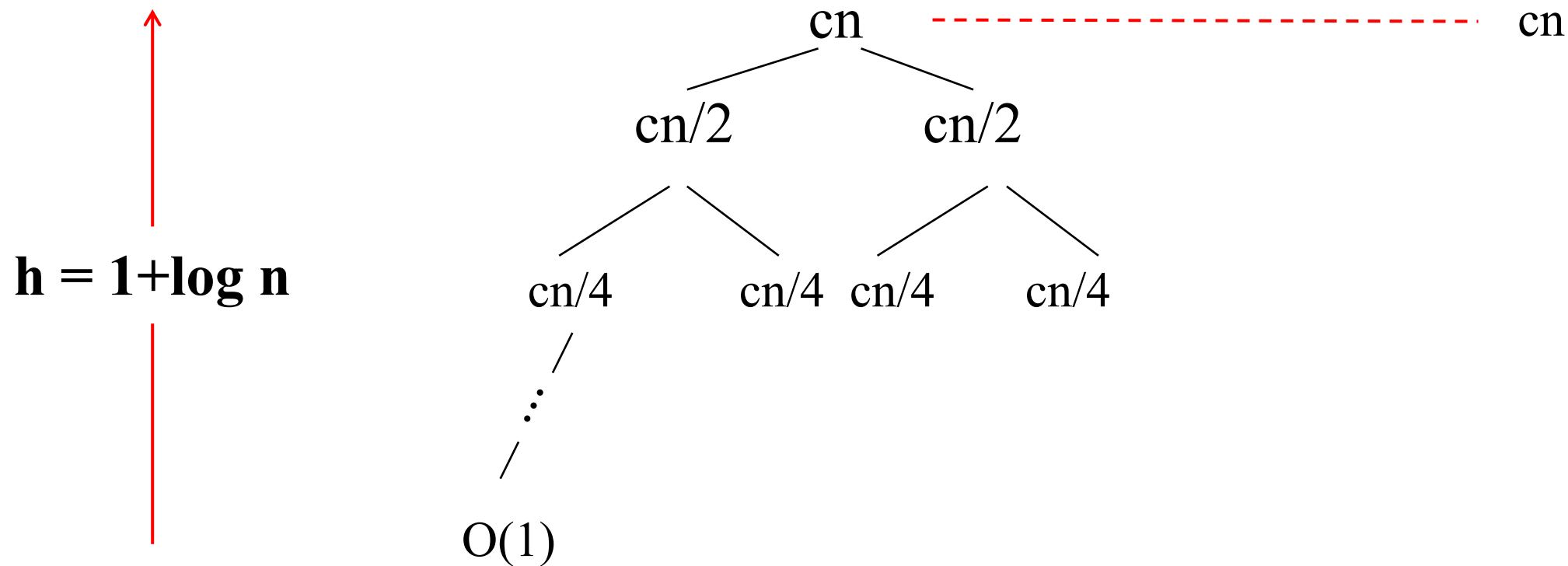
解决 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$ 是常数。



递归树



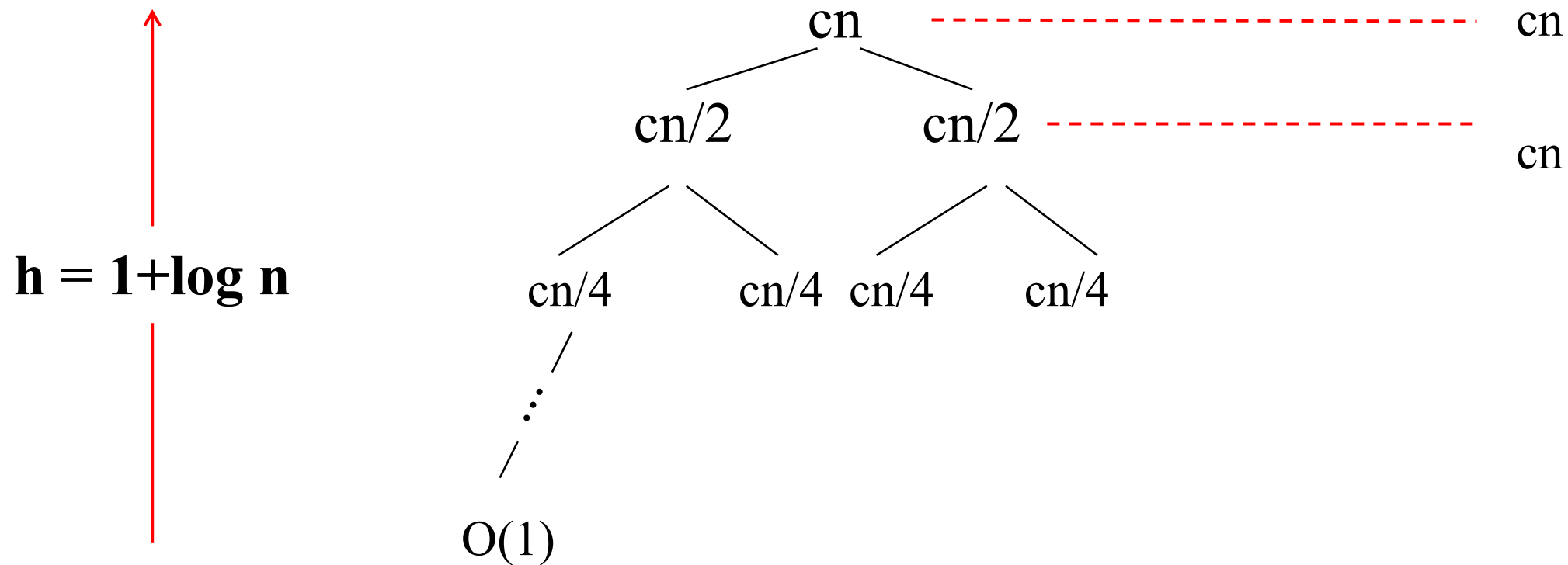
解决 $T(n)=2T(n/2)+cn$ ，其中 $c>0$ 是常数。



递归树



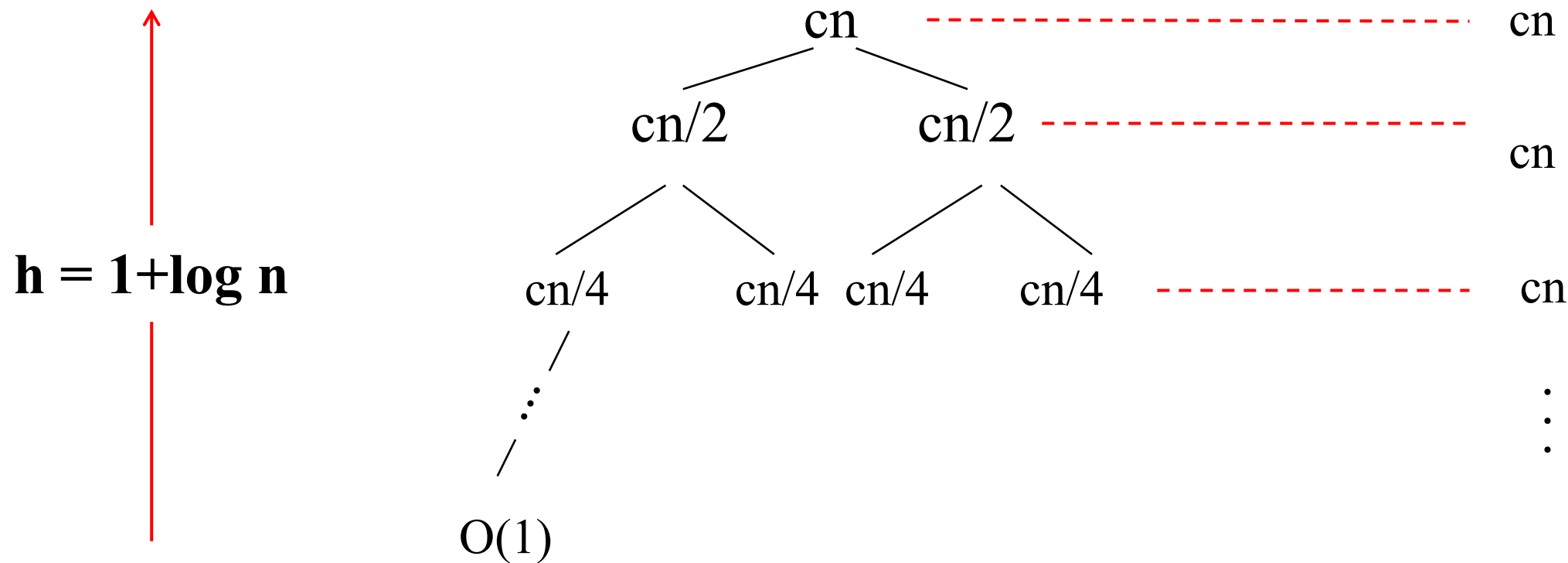
解决 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$ 是常数。



递归树



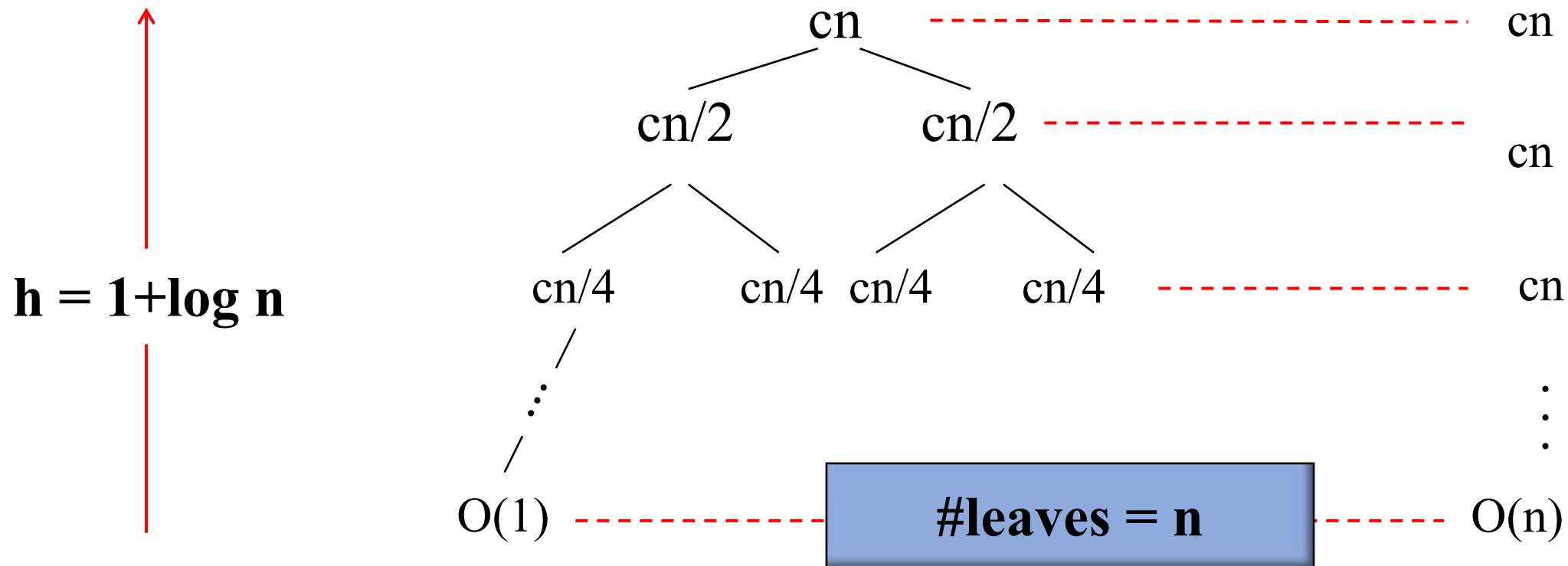
解决 $T(n)=2T(n/2)+cn$, 其中 $c>0$ 是常数。



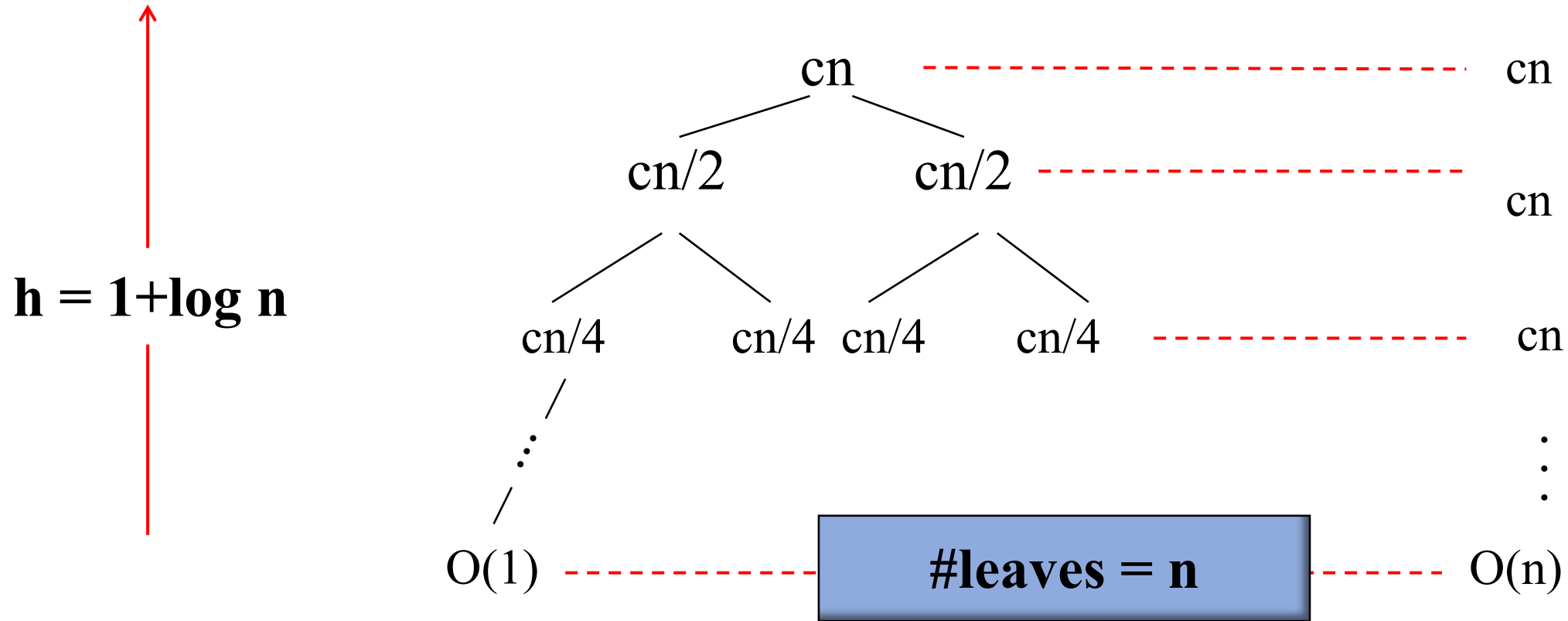
递归树



解决 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$ 是常数。

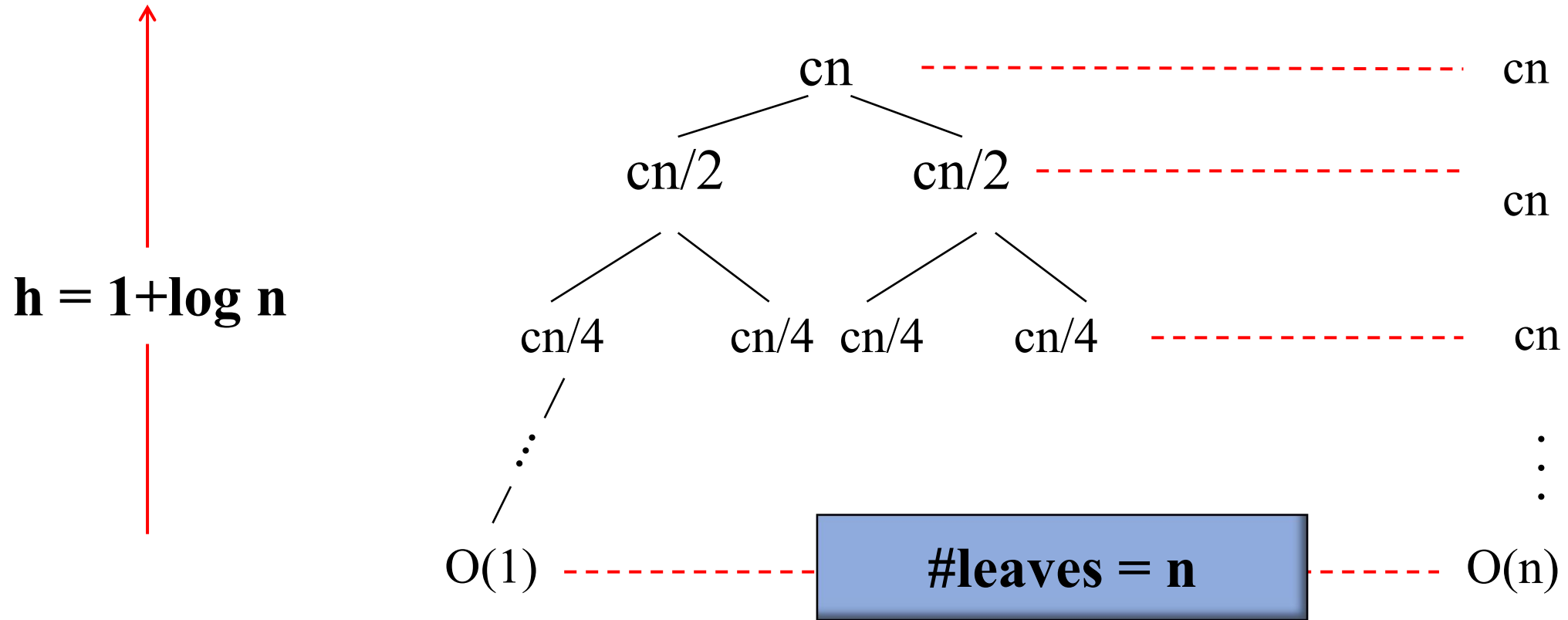


解决 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$ 是常数。



Total ?

解决 $T(n)=2T(n/2)+cn$, 其中 $c>0$ 是常数。



每一层的执行工作量相等。

Total = $O(n \log n)$

快速排序



- 划分：将数组围绕枢轴 x 划分为两个子数组



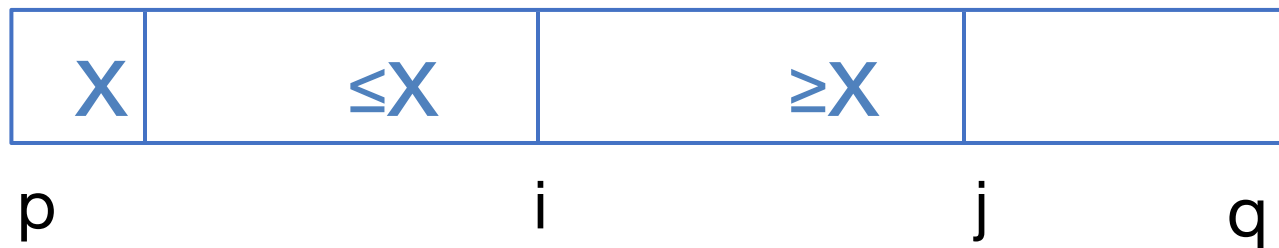
- 治：递归地对子数组进行排序
- 组合：合并两个子数组（这是简单的）

划分



选择 $A[p, \dots, q]$ 中的元素 p 作为枢轴

所有在 $A[p+1, i]$ 中的元素都小于 x ，所有在 $A[i+1, j]$ 中的元素都大于 x ，所有在 $A[j+1, q]$ 中的元素都是未知的



时间复杂度为 $O(n)$



$x = 6$

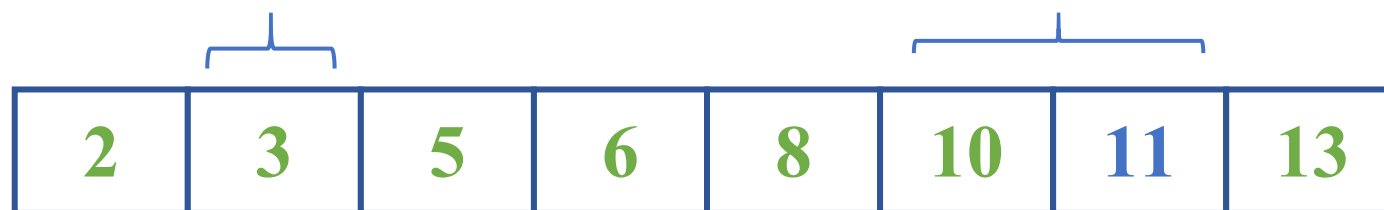
排序子数组



2	5	3	6	8	13	10	11
---	---	---	---	---	----	----	----



2	5	3	6	8	13	10	11
---	---	---	---	---	----	----	----



2	3	5	6	8	10	11	13
---	---	---	---	---	----	----	----



2	3	5	6	8	10	11	13
---	---	---	---	---	----	----	----



2	3	5	6	8	10	11	13
---	---	---	---	---	----	----	----

排序完成

时间复杂度



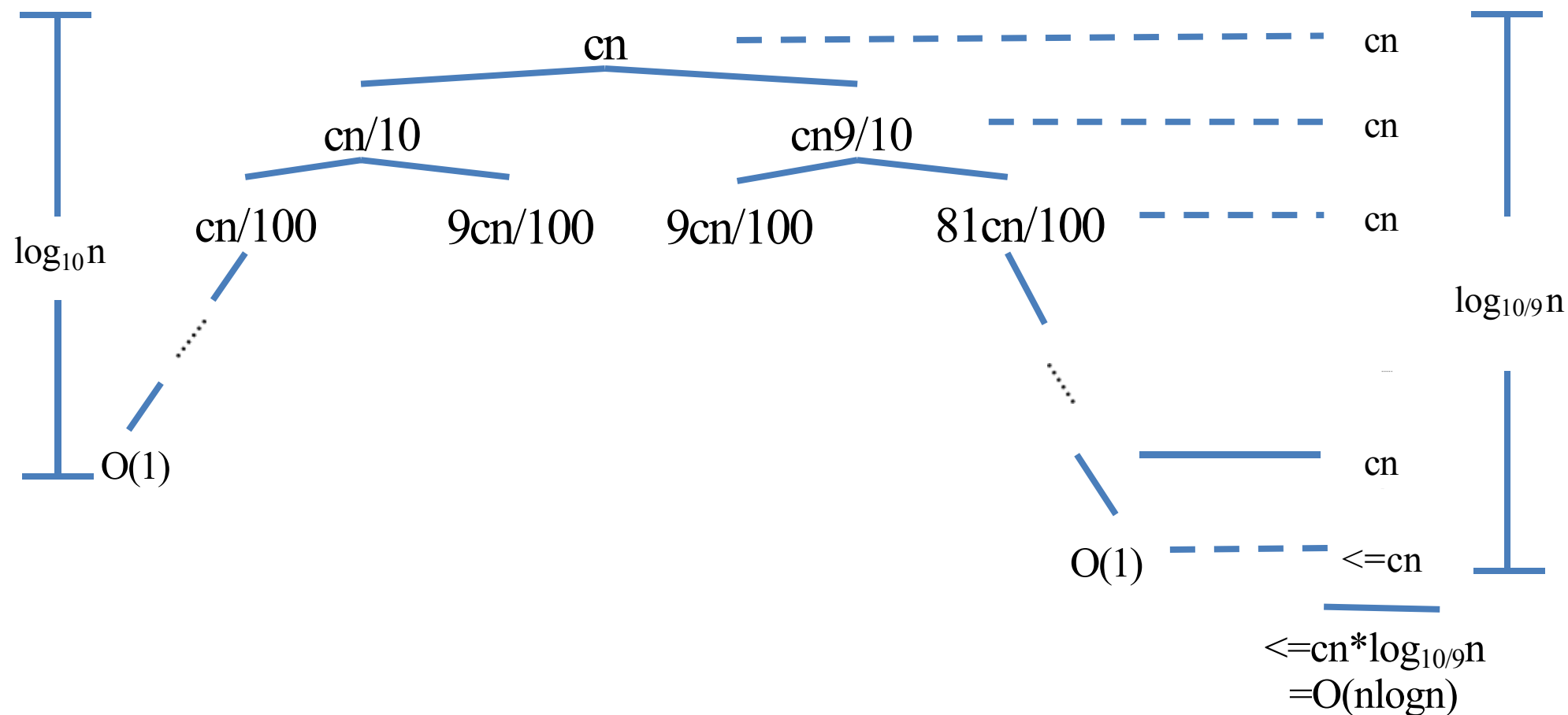
- 最好情况下: $T(n)=2T(n/2)+\Theta(n)=\Theta(n\log n)$
- 最坏情况下: $T(n)=T(0)+T(n-1)+\Theta(n)=\Theta(1)+T(n-1)+\Theta(n)$
 $=T(n-1)+\Theta(n)=\Theta(n^2)$
- 其他情况:

第一种不平衡的情况



分割总是1:9

$$T(n) = T(n/10) + T(9n/10) + O(n)$$



第二个不平衡的情况



- 交替的最佳和最差情况
- 最好情况下: $B(n) = 2W(n/2) + \Theta(n)$
- 最坏情况下: $W(n) = B(n-1) + \Theta(n)$
- 其他情况: $B(n) = 2(B(n/2 - 1) + \Theta(n/2)) + \Theta(n)$
$$= 2B(n/2 - 1) + 2 * \Theta(n)$$
$$= O(n \log n)$$

随机化快速排序



- 随机选择枢轴元素
- 运行时间不依赖于输入的运行顺序

分析

- 我们首先定义一个指示随机变量 x_k
- 如果生成的是 $k:n-k-1$ 的分割, 那么 $x_k=1$, 否则 $x_k=0$
- 于是, $\text{pr}\{x_k=1\}=1/n$
- 然后, x_k 的期望值是

$$E[x_k]=0*\text{pr}\{x_k=0\}+1*\text{pr}\{x_k=1\}=1/n$$

分析

- $T(n) = T(0) + T(n-1) + \Theta(n)$ if $0:n-1$ split
- $T(n) = T(1) + T(n-2) + \Theta(n)$ if $1:n-2$ split
- ...
- $T(n) = T(n-1) + T(0) + \Theta(n)$ if $n-1:0$ split
- Then, $T(n) = \sum_{k=0}^{n-1} x_k \times (T(k) + T(n-k-1) + O(n))$

分析

$$\begin{aligned} E(T(n)) &= E\left(\sum_{k=0}^{n-1} x_k \times (T(k) + T(n-k-1) + O(n))\right) \\ &= \sum_{k=0}^{n-1} E(x_k \times (T(k) + T(n-k-1) + O(n))) \\ &= \frac{1}{n} \sum_{k=0}^{n-1} E((T(k) + T(n-k-1) + O(n))) \\ &= \frac{1}{n} \left(\sum_{k=0}^{n-1} E(T(k)) + \sum_{k=0}^{n-1} E(T(n-k-1)) + \sum_{k=0}^{n-1} O(n) \right) \end{aligned}$$

分析

$$\begin{aligned} E(T(n)) &= \frac{1}{n} \sum_{k=0}^{n-1} E(T(k)) + \frac{1}{n} \sum_{k=0}^{n-1} E(T(n-k-1)) + \\ &\frac{1}{n} \sum_{k=0}^{n-1} O(n) \\ &= \frac{2}{n} \sum_{k=0}^{n-1} E(T(k)) + \frac{1}{n} \sum_{k=0}^{n-1} O(n) \\ &= \frac{2}{n} \sum_{k=0}^{n-1} E(T(k)) + O(n) \end{aligned}$$

吸收 $k=0$ 和 $k=1$ ，我们可以得到一个更简单的公式：

$$E(T(n)) = \frac{2}{n} \sum_{k=2}^{n-1} E(T(k)) + O(n)$$

分析

为证明 $E(T(n))=O(n\log n)$, 考虑下述引理

$$\sum_{k=2}^{n-1} k \log k \leq \int_2^{n-1} k \times \log k \, dk$$

$$\text{因为 } \int k \times \log k \, dk = \frac{1}{2} k^2 \times \log k - \frac{1}{4 \times \ln 2} k^2 + C$$

$$\text{所以 } \sum_{k=2}^{n-1} k \log k \leq \frac{1}{2} n^2 \times \log n$$

分析

定理： 对于 $E(T(n)) = \frac{2}{n} \sum_{k=2}^{n-1} E(T(k)) + O(n)$ ，我们可以通过归纳法证明 $E(T(n))=O(n \log n)$

证明： 数学归纳，基础步： $n=1$ 的时候，显然成立；

归纳步： $n < m$ 的时候都成立，然后证明 $n = m$ 的时候成立

$$\begin{aligned} E(T(m)) &= \frac{2}{n} \sum_{k=2}^{n-1} E(T(k)) + O(n) = \frac{2}{n} \sum_{k=2}^{n-1} O(k \log k) + O(n) \\ &\leq c_1 \frac{2}{n} \sum_{k=2}^{n-1} k \log k + c_2 n \leq c_1 \times \frac{2}{n} \times \frac{1}{2} n^2 \log n + c_2 n = c_1 n \log n + c_2 n \\ &= O(n \log n) \end{aligned}$$

比较排序



我们迄今为止看到的所有排序算法都是**比较排序**：它们只使用比较来确定元素的相对顺序。

我们看到过的最好的运行时间是 $O(n \log n)$ 。

$O(n \log n)$ 是我们能做到的最好吗？

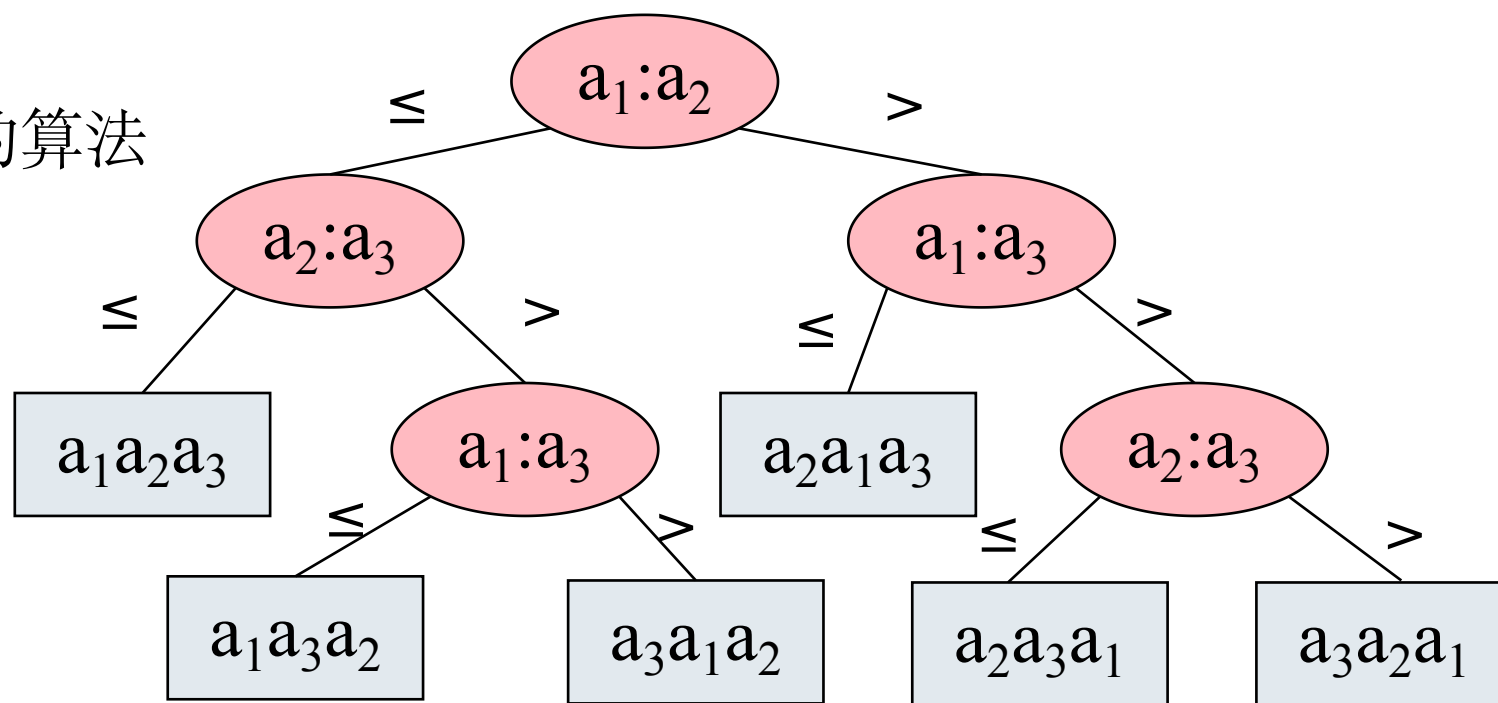
决策树



对 n 个字符 $\langle a_1, a_2, \dots, a_n \rangle$ 排序的算法

节点是两个元素的比较,
例 $a_i:a_j$

分支走向由比对结果决定

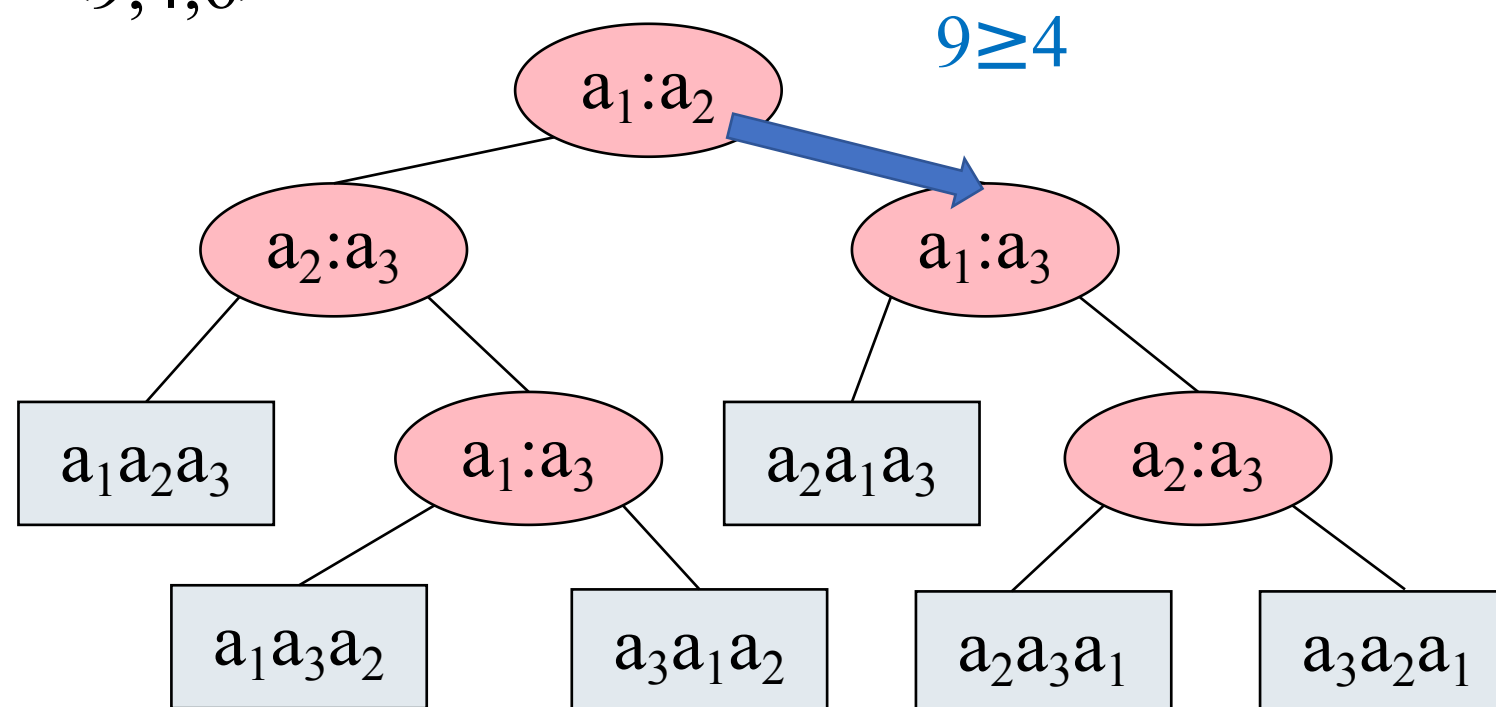


叶节点是标记好的排序, 对应的是排序的结果

决策树例子



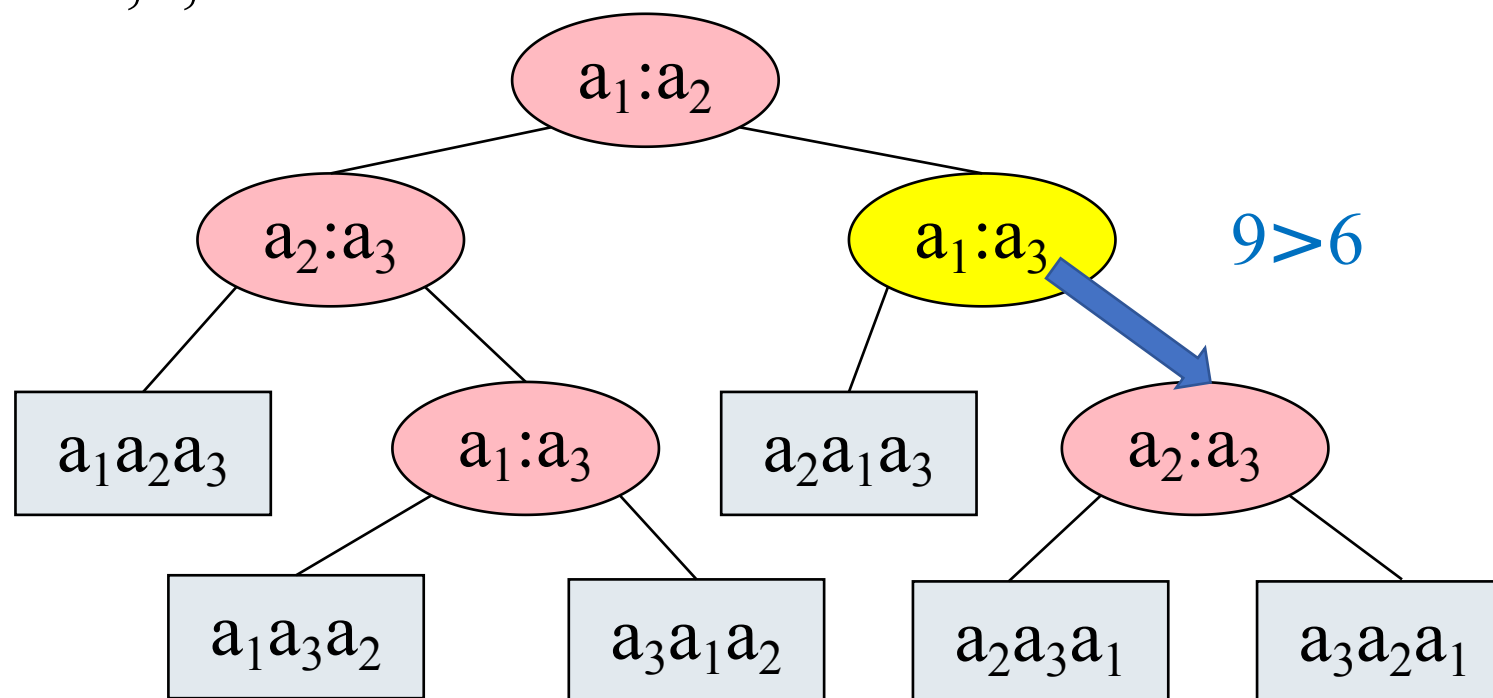
$\text{Sort}\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$



决策树例子



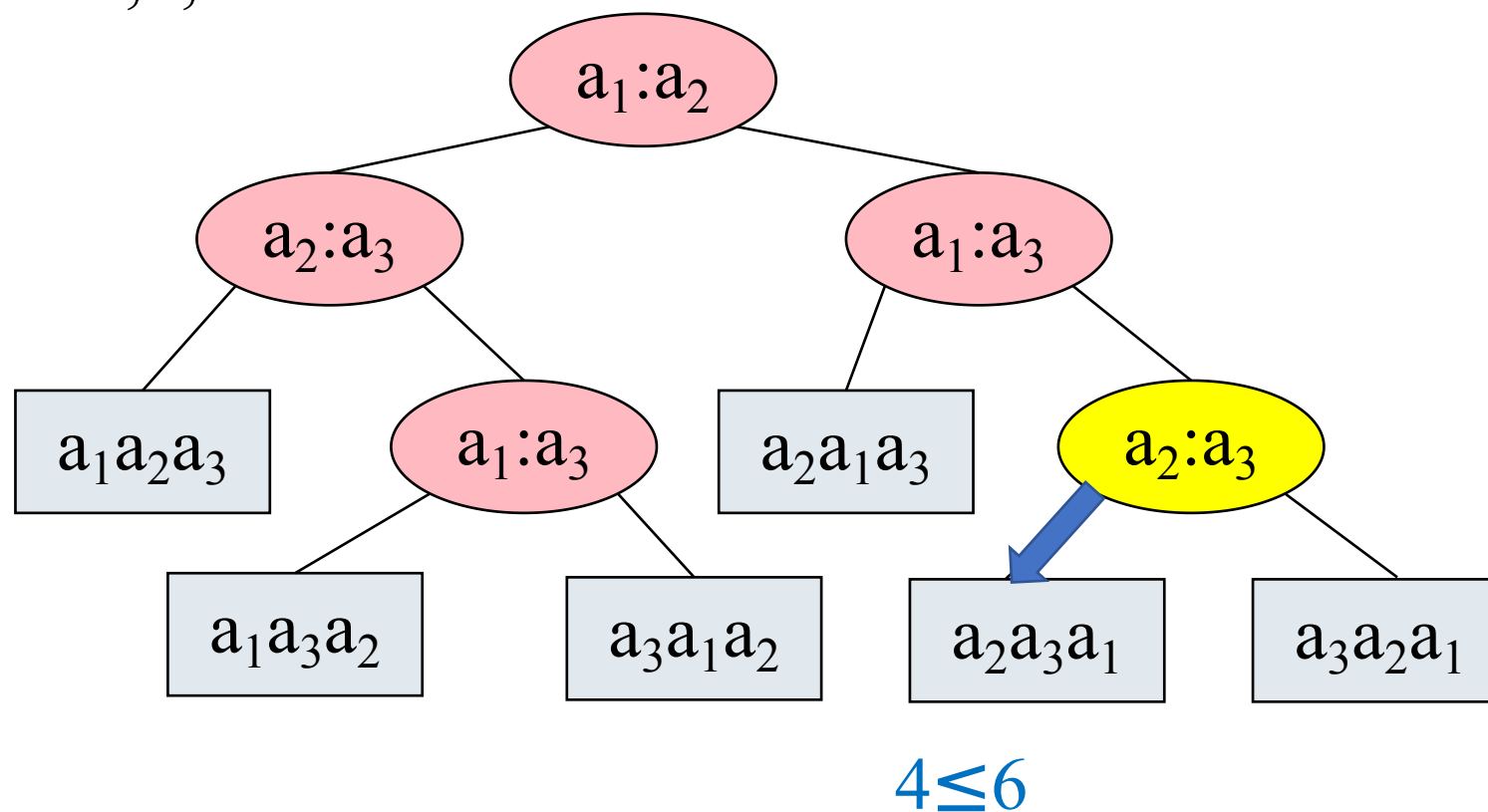
$\text{Sort}\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$



决策树例子



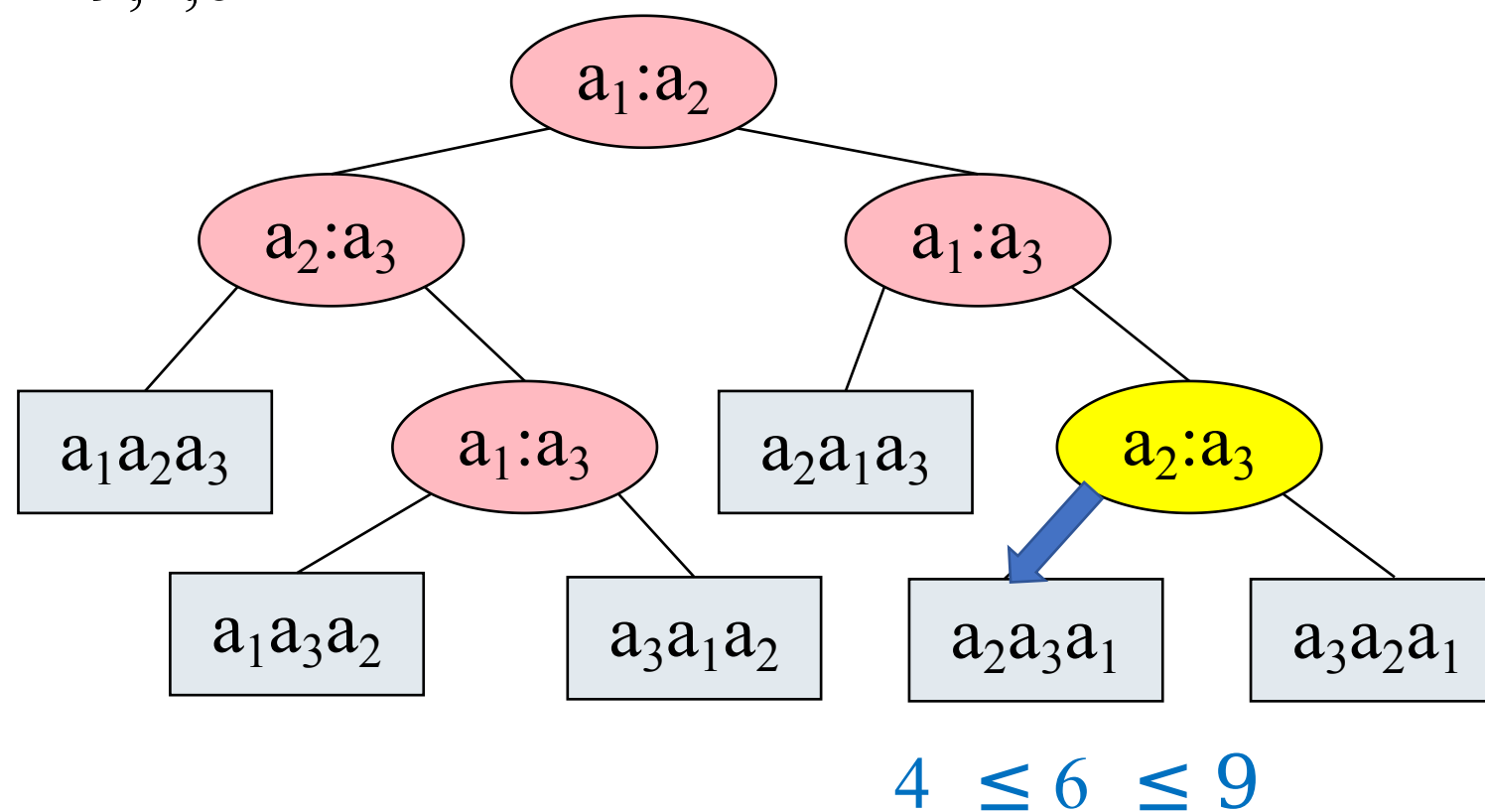
$\text{Sort}\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$



决策树例子



$\text{Sort}\langle a_1, a_2, a_3 \rangle = \langle 9, 4, 6 \rangle$



决策树模型



一个决策树可以模拟任何比对排序的执行

- 输入的大小为 n
- 一个从跟节点到叶节点的路径表示这个算法可能执行的一系列比对的跟踪
- 算法的运行时间=路径的长度
- 算法的最差情况下的运行时间=树的高度

决策树排序的下界



定理：任何用于 n 个元素的决策树的高度必须满足 $\Omega(n \log n)(\geq)$ 。

证明：首先，由于存在 $n!$ 种可能的排列，因此树必须包含 $(\geq)n!$ 个叶子节点。

一个高度为 h 的二叉树最多有 $\leq 2^h$ 个叶子节点。为了能够对其进行排序，必须满足以下条件：

$$2^h \geq n!$$

$$h \geq \log(n!)$$

$$\geq \log((n/e)^n)$$

$$= n \log n - n \log e$$

$$= \Omega(n \log n)$$

目录

第一节

比较排序

第二节

线性时间排序

线性时间排序



计数排序：元素之间没有比对

- 输入数组的值在1-k之间
- 输出数组B是对数组A的排序
- 辅助存储空间，长度为k数组C
- 运行时间为 $O(n+k)$

计数排序



$n=5, k=4$

	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:				

数组C的索引对应的数组A
中每一个可能的键值

循环1: 初始化



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	0	0	0	0

初始化数组C的每一个元素
值为0

```
for i ← 1 to k  
  do C[ i ] ← 0
```

循环2: 计算频率



	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	0	0	0	1

$C[i]$ = 数组A中键值为i的元素个数, $i=4$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

循环2: 计算频率



	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	0	1

$C[i]$ = 数组A中键值为i的元素个数, $i=1$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

循环2: 计算频率



	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	1	1

$C[i]$ = 数组A中键值为i的元素个数, $i=3$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

循环2: 计算频率



	1	2	3	4	5
A:	4	1	3	4	3
B:					

	1	2	3	4
C:	1	0	1	2

$C[i]$ = 数组A中键值为i的元素个数, $i=4$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

循环2: 计算频率



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	1	0	2	2

$C[i]$ = 数组A中键值为i的元素个数, $i=3$

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

A parenthesis:快速完成



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	1	0	2	2

对频率数组C从头到尾走一遍，我们可以排序好的键值放到输出数组B中来。

for $j \leftarrow 1$ **to** n

do $C[A[j]] \leftarrow C[A[j]] + 1$

► $C[i] = |\{\text{key} = i\}|$

A parenthesis:快速完成



	1	2	3	4	5
A:	4	1	3	4	3

B:	1				
----	---	--	--	--	--

	1	2	3	4
C:	1	0	2	2

对于 $C[i]$, 数组A中键值为 i 的元素数目为 $C[i]$

对于 $C[1]$, 数组A中键值为1的元素数目为1

A parenthesis:快速完成



	1	2	3	4	5
A:	4	1	3	4	3

B:	1				
----	---	--	--	--	--

	1	2	3	4
C:	1	0	2	2

对于C[2], 数组A中键值为2的元素数目为0!

A parenthesis:快速完成



	1	2	3	4	5
A:	4	1	3	4	3

B:	1	3	3		
----	---	---	---	--	--

	1	2	3	4
C:	1	0	2	2

对于C[3], 数组A中键值为3的元素数目为2!

A parenthesis:快速完成



	1	2	3	4	5
A:	4	1	3	4	3

B:	1	3	3	4	4
----	---	---	---	---	---

	1	2	3	4
C:	1	0	2	2

对于C[4], 数组A中键值为4的元素数目为2!

B是排好了序, 但只对键值排了序, 没有对A中的元素排序, 因为A中元素除了键值还有其他的数据, 这里B只有键值的排序而没有对A中元素的交换!

循环3: 从频率到累积频率.....



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	1	0	2	2

C':	1	1	2	2
-----	---	---	---	---

$C[i]$ = 数组A中键值 $\leq i$ 的元素个数, $i=2$

for $i \leftarrow 2$ to k

do $C[i] \leftarrow C[i] + C[i - 1]$ $\blacktriangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环3: 从频率到累积频率.....



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	1	0	2	2

C':	1	1	3	2
-----	---	---	---	---

$C[i]$ = 数组A中键值 $\leq i$ 的元素个数, $i=3$

for $i \leftarrow 2$ to k

do $C[i] \leftarrow C[i] + C[i-1]$ $\blacktriangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环3: 从频率到累积频率.....



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C:	1	0	2	2

C':	1	1	3	5
-----	---	---	---	---

$C[i]$ = 数组A中键值 $\leq i$ 的元素个数, $i=4$

for $i \leftarrow 2$ to k

do $C[i] \leftarrow C[i] + C[i - 1]$ $\blacktriangleright$ $C[i] = |\{\text{key} \leq i\}|$

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C	1	1	3	5
'				
:				

for $j \leftarrow n$ downto 1

do $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$

对数组A从尾到头进行元素置换

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:					
----	--	--	--	--	--

	1	2	3	4
C' :	1	1	3	5

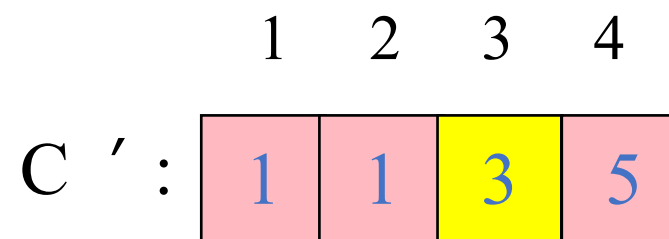
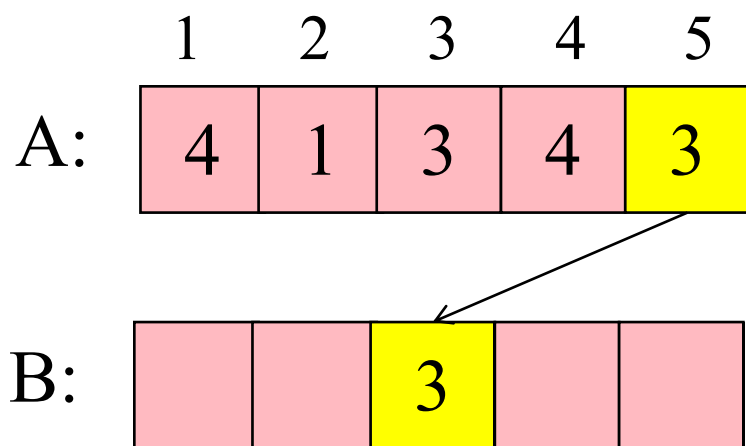
for $j \leftarrow n$ downto 1

do $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$

正好有3个元素 $\leq A[5]$, 那么应该把
A[5]元素放到哪个位置去?

循环 4: 交换 A 中的元素



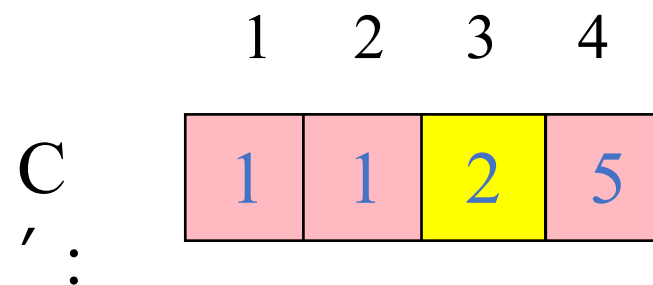
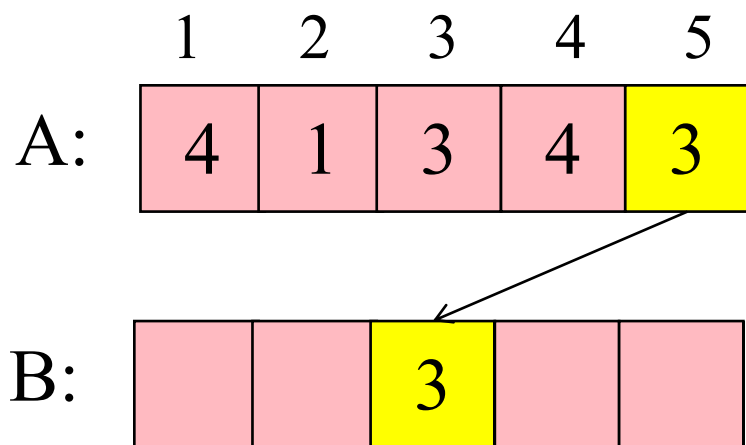
```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[5]放到B[C[A[5]]]中

用了C[A[5]]的最上面一个元素位置

更新C[A[5]]来给A数组中其他键值为A[5]
元素来安排位置!

循环 4: 交换 A 中的元素



```
for  $j \leftarrow n$  downto 1  
do  $B[C[A[j]]] \leftarrow A[j]$   
    $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

把A[5]放到B[C[A[5]]]中

用了C[A[5]]的最上面一个元素位置

更新C[A[5]]来给A数组中其他键值为A[5]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:			3		
----	--	--	---	--	--

	1	2	3	4
C' :	1	1	2	5

```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[4]放到B[C[A[4]]]中

用了C[A[4]]的最上面一个元素位置

更新C[A[4]]来给A数组中其他键值为A[4]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:			3		
----	--	--	---	--	--

	1	2	3	4
C	1	1	2	5
:				

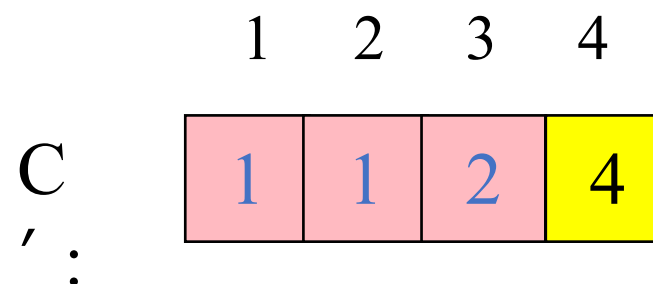
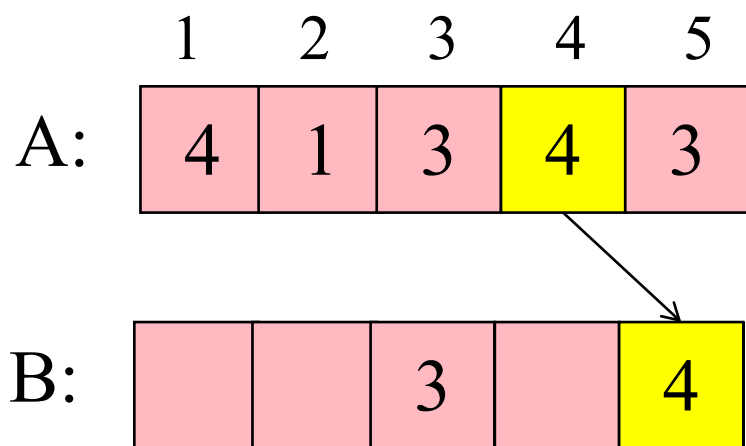
```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[4]放到B[C[A[4]]]中

用了C[A[4]]的最上面一个元素位置

更新C[A[4]]来给A数组中其他键值为A[4]
元素来安排位置!

循环 4: 交换 A 中的元素



```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[4]放到B[C[A[4]]]中

用了C[A[4]]的最上面一个元素位置

更新C[A[4]]来给A数组中其他键值为A[4]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:			3		4
----	--	--	---	--	---

	1	2	3	4
C' :	1	1	2	4

```
for  $j \leftarrow n$  downto 1  
do  $B[C[A[j]]] \leftarrow A[j]$   
    $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

把A[3]放到B[C[A[3]]]中

用了C[A[3]]的最上面一个元素位置

更新C[A[3]]来给A数组中其他键值为A[4]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:			3		4
----	--	--	---	--	---

	1	2	3	4
C' :	1	1	2	4

for $j \leftarrow n$ **downto** 1

do $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$

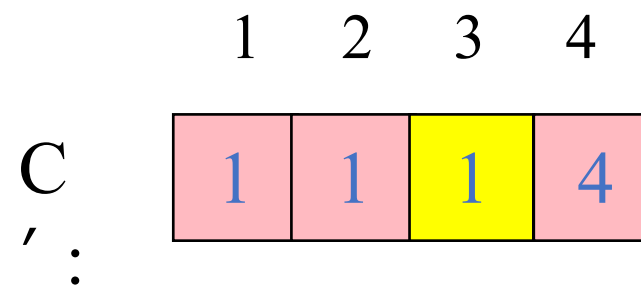
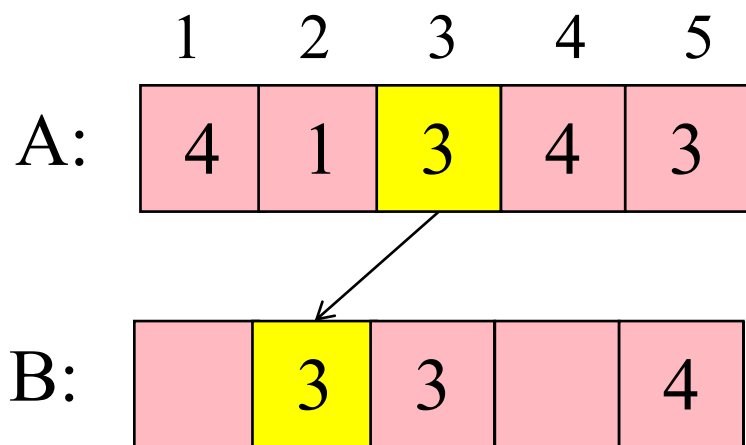
把A[3]放到B[C[A[3]]]中

用了C[A[3]]的最上面一个元素位置

更新C[A[3]]来给A数组中其他键值为A[4]

元素来安排位置!

循环 4: 交换 A 中的元素



```
for j ← n downto 1
do B[ C[ A[ j ] ] ] ← A[ j ]
   C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[3]放到B[C[A[3]]]中

用了C[A[3]]的最上面一个元素位置

更新C[A[3]]来给A数组中其他键值为A[4]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:		3	3		4
----	--	---	---	--	---

	1	2	3	4
C' :	1	1	1	4

```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

把A[2]放到B[C[A[2]]]中

用了C[A[2]]的最上面一个元素位置

更新C[A[2]]来给A数组中其他键值为A[2]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:		3	3		4
----	--	---	---	--	---

	1	2	3	4
C' :	1	1	1	4

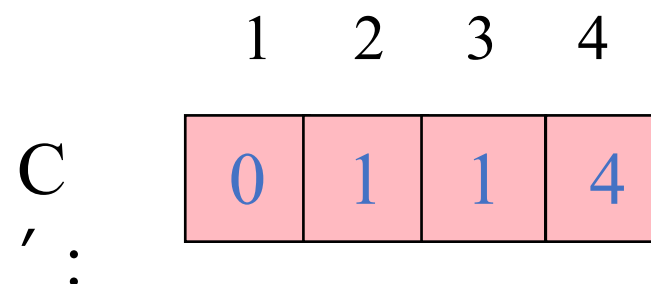
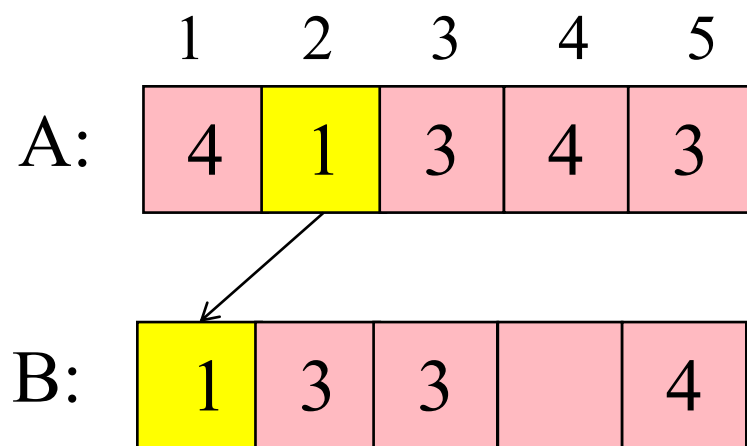
```
for  $j \leftarrow n$  downto 1  
  do  $B[C[A[j]]] \leftarrow A[j]$   
      $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

把A[2]放到B[C[A[2]]]中

用了C[A[2]]的最上面一个元素位置

更新C[A[2]]来给A数组中其他键值为A[2]
元素来安排位置!

循环 4: 交换 A 中的元素



```
for j ← n downto 1
do B[ C[ A[ j ] ] ] ← A[ j ]
   C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[2]放到B[C[A[2]]]中

用了C[A[2]]的最上面一个元素位置

更新C[A[2]]来给A数组中其他键值为A[2]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:	1	3	3		4
----	---	---	---	--	---

	1	2	3	4
C	0	1	1	4
'	:			

```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[1]放到B[C[A[1]]]中

用了C[A[1]]的最上面一个元素位置

更新C[A[1]]来给A数组中其他键值为A[1]
元素来安排位置!

循环 4: 交换 A 中的元素



	1	2	3	4	5
A:	4	1	3	4	3

B:	1	3	3		4
----	---	---	---	--	---

	1	2	3	4
C	0	1	1	4
'	:			

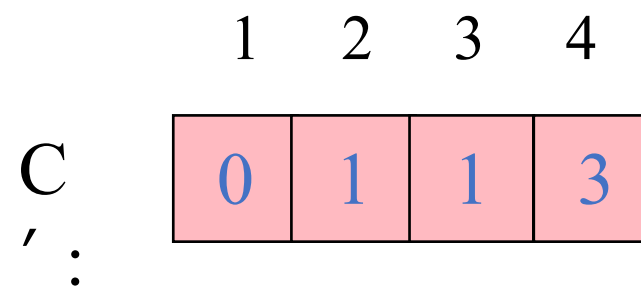
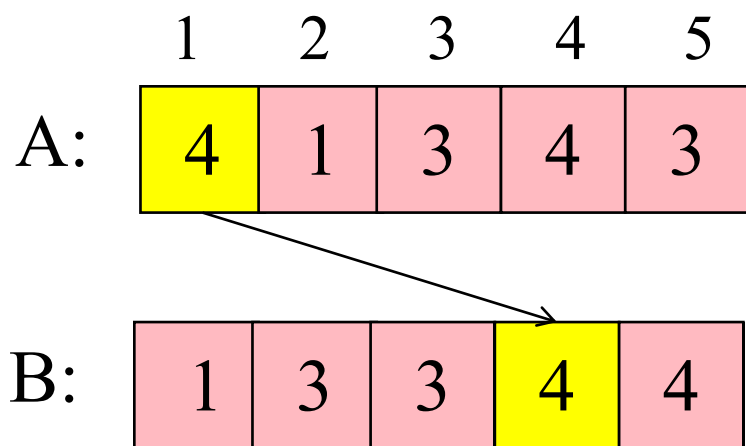
```
for  $j \leftarrow n$  downto 1  
do  $B[C[A[j]]] \leftarrow A[j]$   
    $C[A[j]] \leftarrow C[A[j]] - 1$ 
```

把A[1]放到B[C[A[1]]]中

用了C[A[1]]的最上面一个元素位置

更新C[A[1]]来给A数组中其他键值为A[1]
元素来安排位置!

循环 4: 交换 A 中的元素



```
for j ← n downto 1
  do B[ C[ A[ j ] ] ] ← A[ j ]
     C[ A[ j ] ] ← C[ A[ j ] ] - 1
```

把A[1]放到B[C[A[1]]]中

用了C[A[1]]的最上面一个元素位置

更新C[A[1]]来给A数组中其他键值为A[1]
元素来安排位置!

计数排序



for $i \leftarrow 1$ to k

do $C[i] \leftarrow 0$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

for $i \leftarrow 2$ to k

do $C'[i] \leftarrow C[i] + C[i - 1]$

for $j \leftarrow n$ downto 1

do $B[C'[A[j]]] \leftarrow A[j]$

$C'[A[j]] \leftarrow C'[A[j]] - 1$

将数组A中不同键的频率存储在C中。
 $C[i] = |\{\text{key} = i\}|$

将数组A中不同键的频率存储在C中。
 $C[i] = |\{\text{key} \leq i\}|$

使用累积频率构建有序排列

$O(k)$

$O(n)$

$O(k)$

$O(n)$

$O(n+k)$

运行时间



加入 $k=O(n)$,那么计数排序需要

如果 $k=O(n)$,那么计数排序需要的时间是 $O(n)$ 。

但是,排序需要的时间是 $\Omega(n \lg n)$!

哪里出了问题?

答案: 比较排序需要的时间是 $\Omega(n \lg n)$ 。

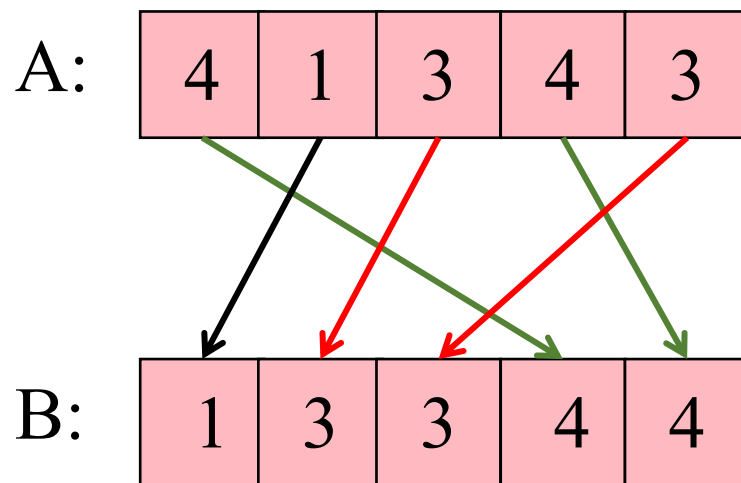
计数排序不是一种比较排序。

实际上, 没有进行过任何元素之间的比较!

稳定排序



计数排序是一种稳定排序：它给同样键值的元素们保留了原先排序！

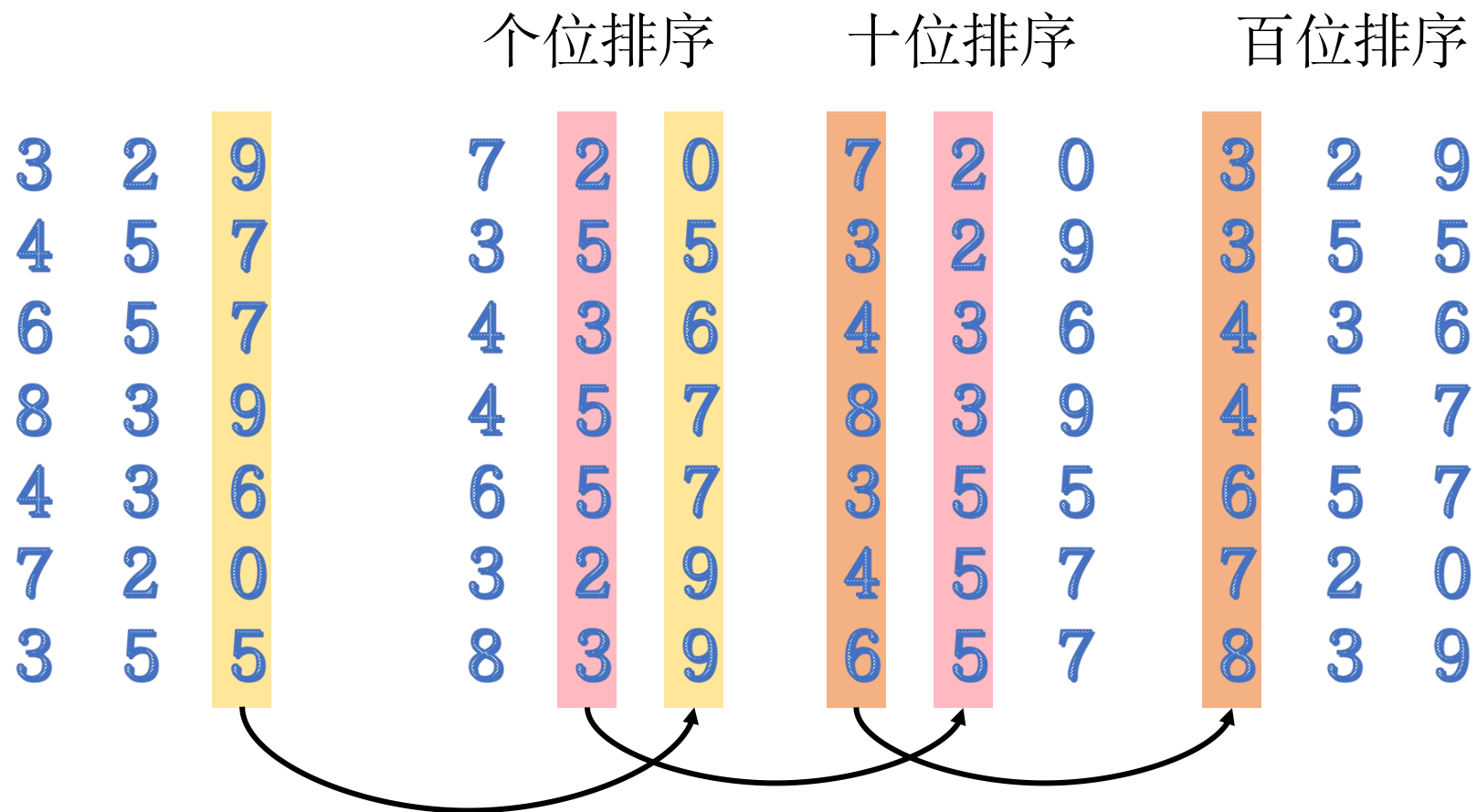


基数排序



- 1890年用来解决美国的人口普查!
- 一个位数接一个位数的排序!
- 原来的（差的）想法：高位优先
- 好想法：低位优先

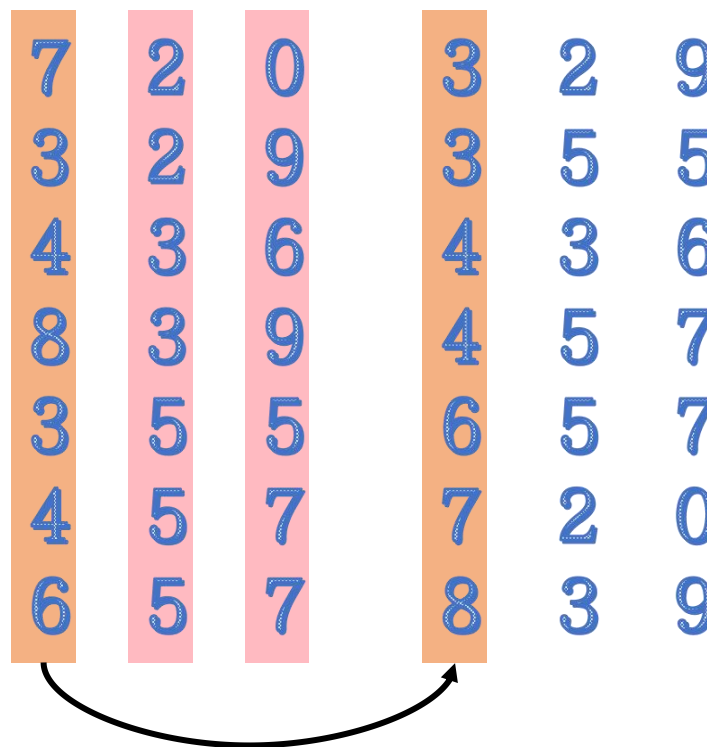
基数排序的操作过程



基数排序的正确性



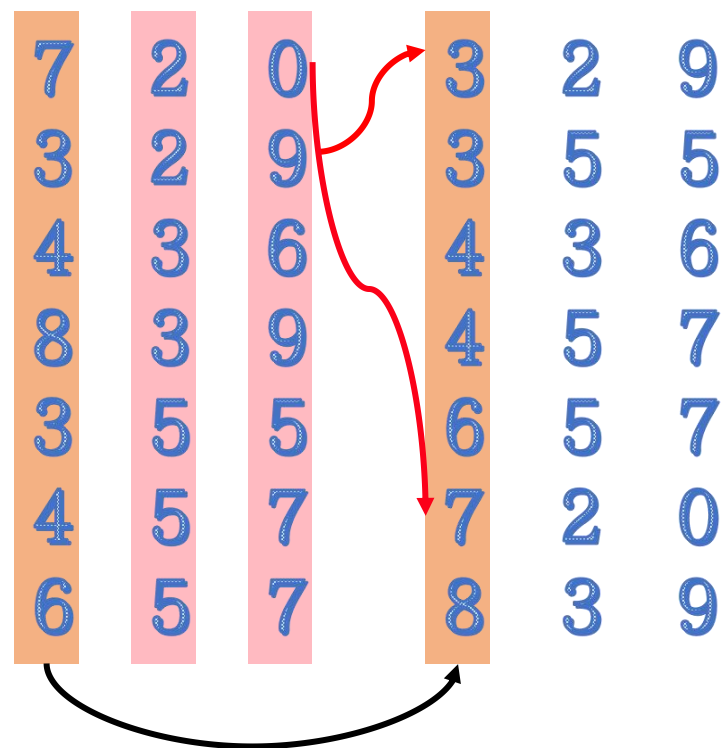
- 对位数上的数学归纳
- 假设 $t-1$ 位数们，都是排好序的
- 对 t 位数的排序操作能保证键值只有 t 位的数组的排序是正确的。



基数排序的正确性



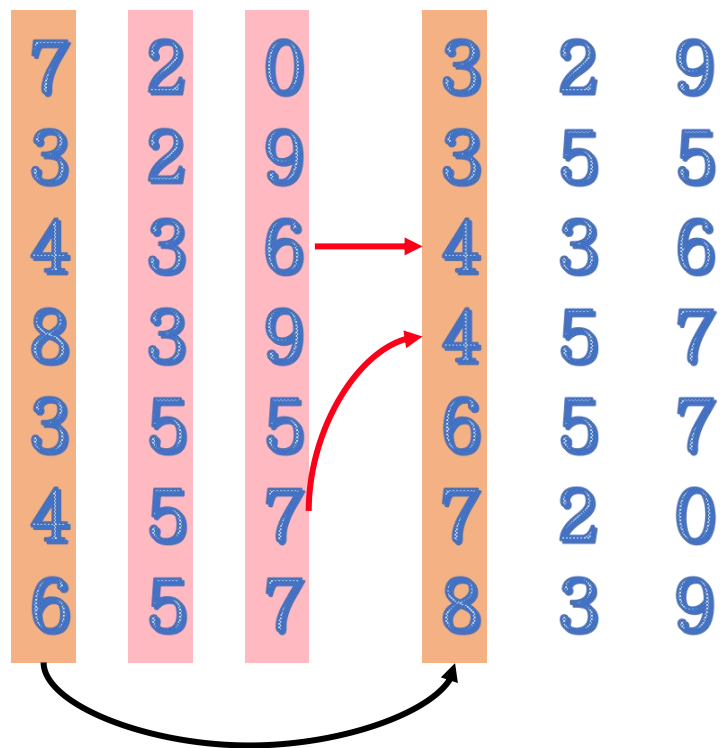
- 对位数上的数学归纳
- 假设 $t-1$ 位数们，都是排好序的
- 任意两个在 t 位数上键值不一样 的元素，在 t 位上比对交换大小 排序后，与这两个元素在数位 $1-t$ 位上的大小顺序是一致的。



基数排序的正确性



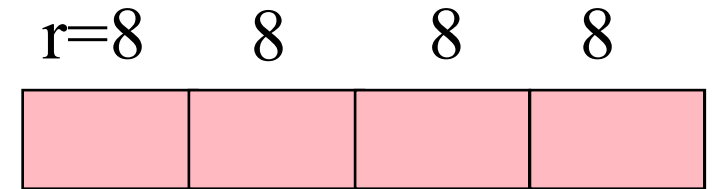
- 对位数上的数学归纳
- 假设 $t-1$ 位数们，都是排好序的
- 任意两个在 t 位数上键值一样的元素，与这两个元素在数位 $1-t$ 位上的大小顺序依然是一致的（得证！）



基数排序的运行时间分析



- 计数排序作为辅助的稳定排序方法。
- 每一个键值由 b 个二进制组成
- 基数为 2^r 个比特，一个键值可以化成 b/r 个位数
- 例如，对于一个32位的字来说，如果我们将每个 b 位的字拆分成 r 位的片段，那么每次计数排序的过程需要 $O(n+2^r)$ 的时间。因此，总体时间为 $(b/r)(n+2^r)$ 。如果我们设定 $r=\log n$ ，那么在每次传递过程中可以得到 $O(n)$ 的时间复杂度，或者总的时间复杂度为 $O(n * b / \log n)$ 。





湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——